



Kubernetes Architecture



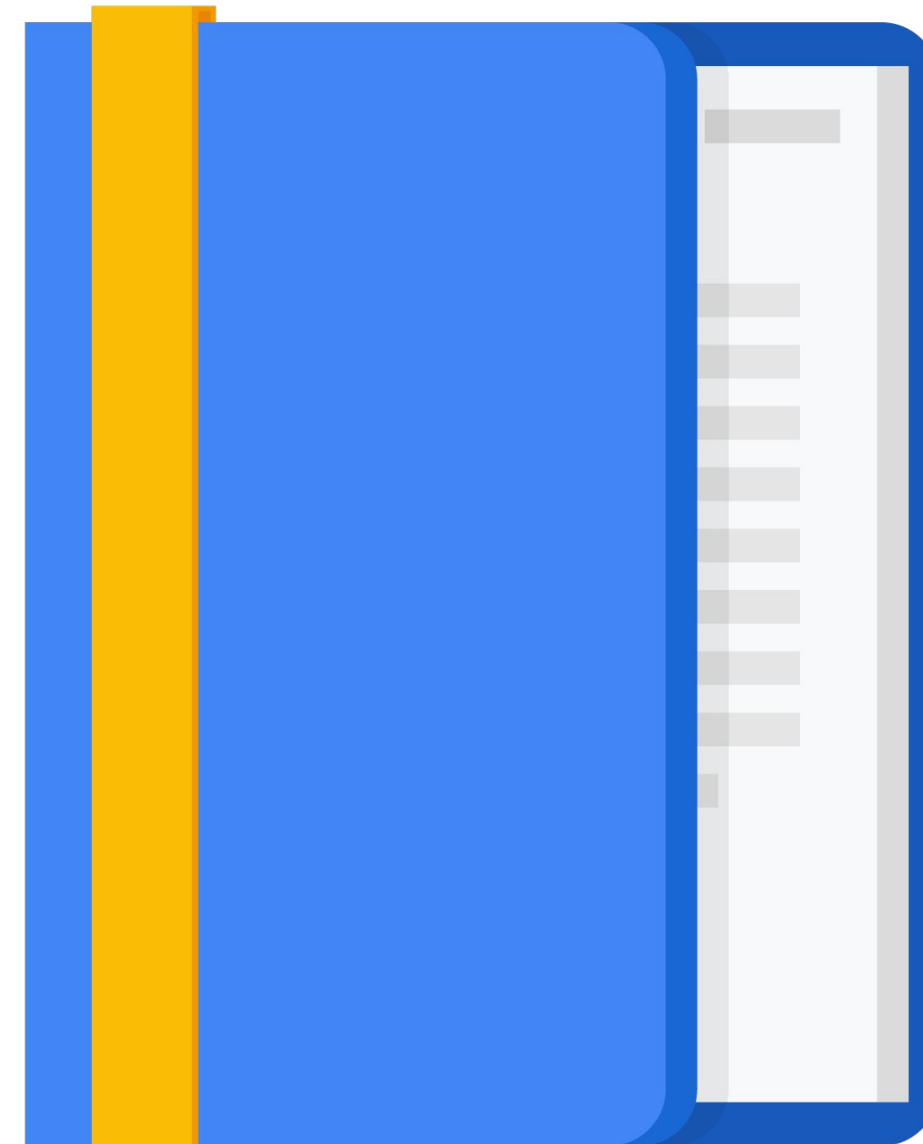
Learn how to ...

Understand Kubernetes objects and the Kubernetes control plane.

Deploy a Kubernetes cluster using Google Kubernetes Engine (GKE).

Deploy Pods to a GKE cluster.

View and manage Kubernetes objects.



Agenda

Kubernetes Concepts

Kubernetes Components

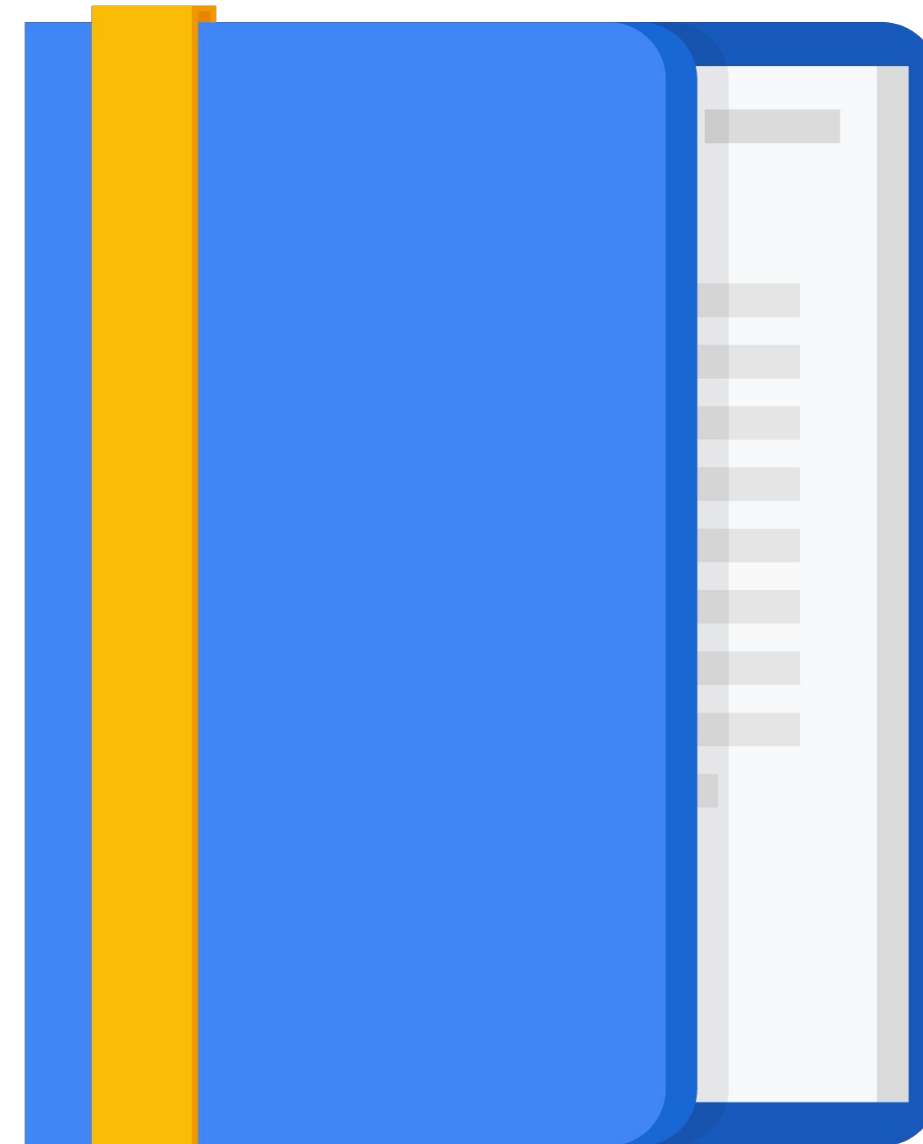
Google Kubernetes Engine Concepts

Object Management

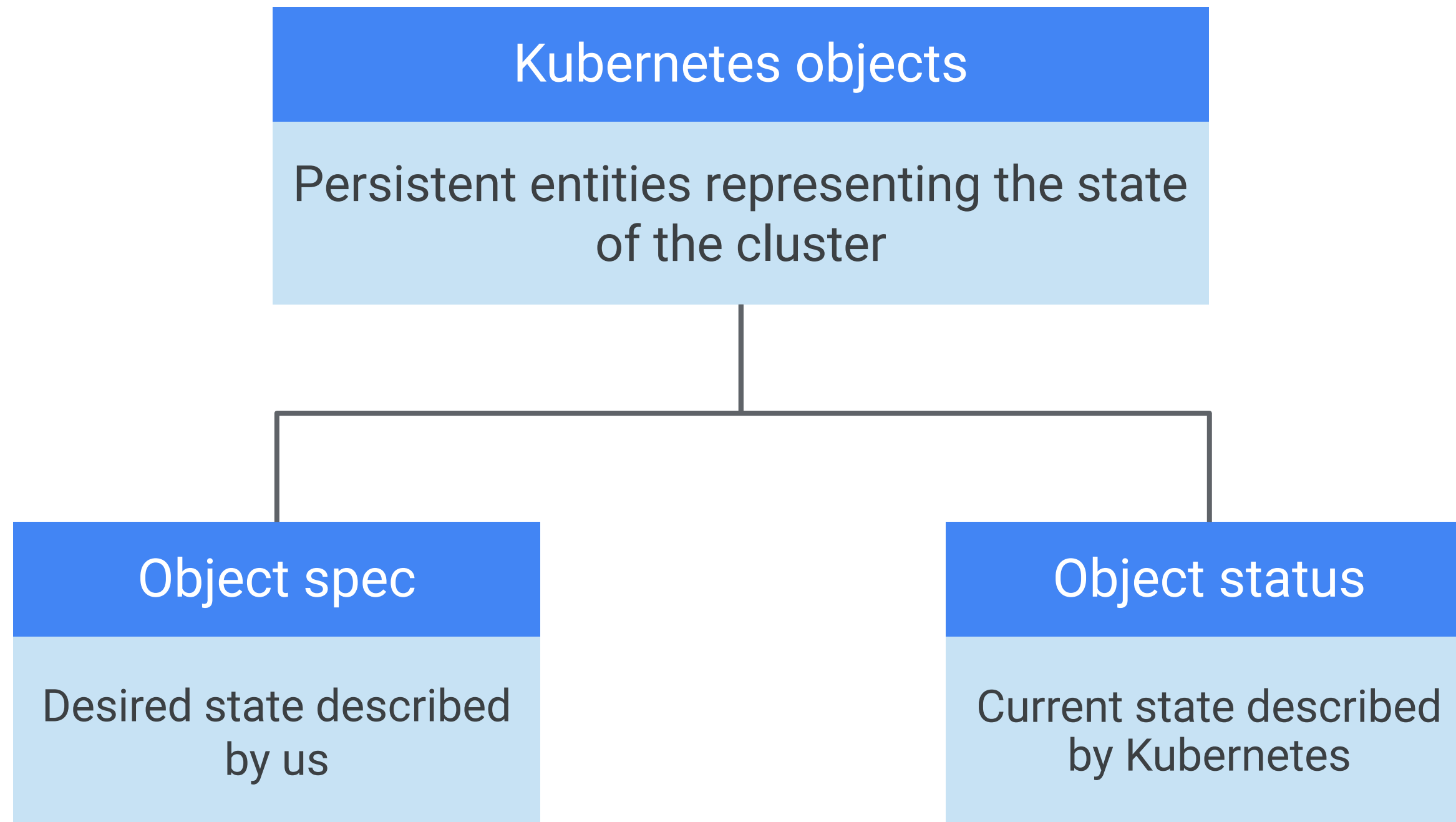
Lab: Deploying Google Kubernetes
Engine

Migrate for Anthos

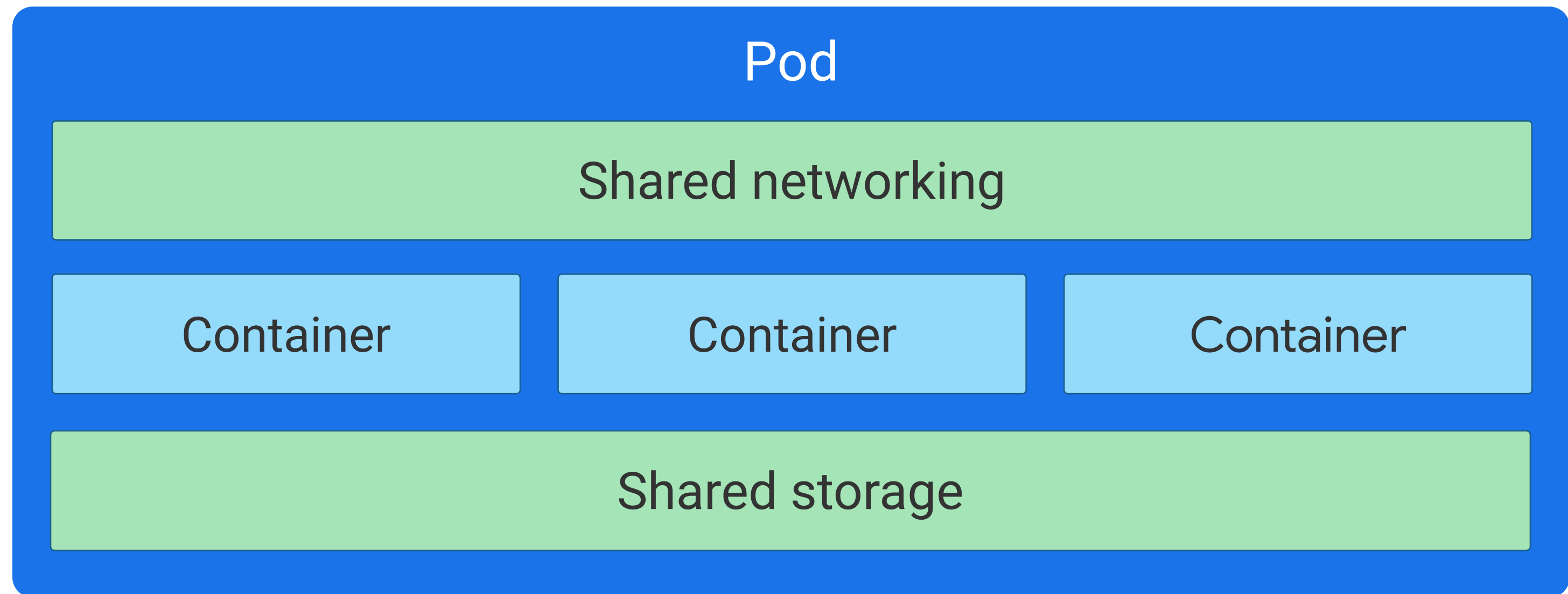
Summary



There are two elements to Kubernetes objects



Containers in a Pod share resources



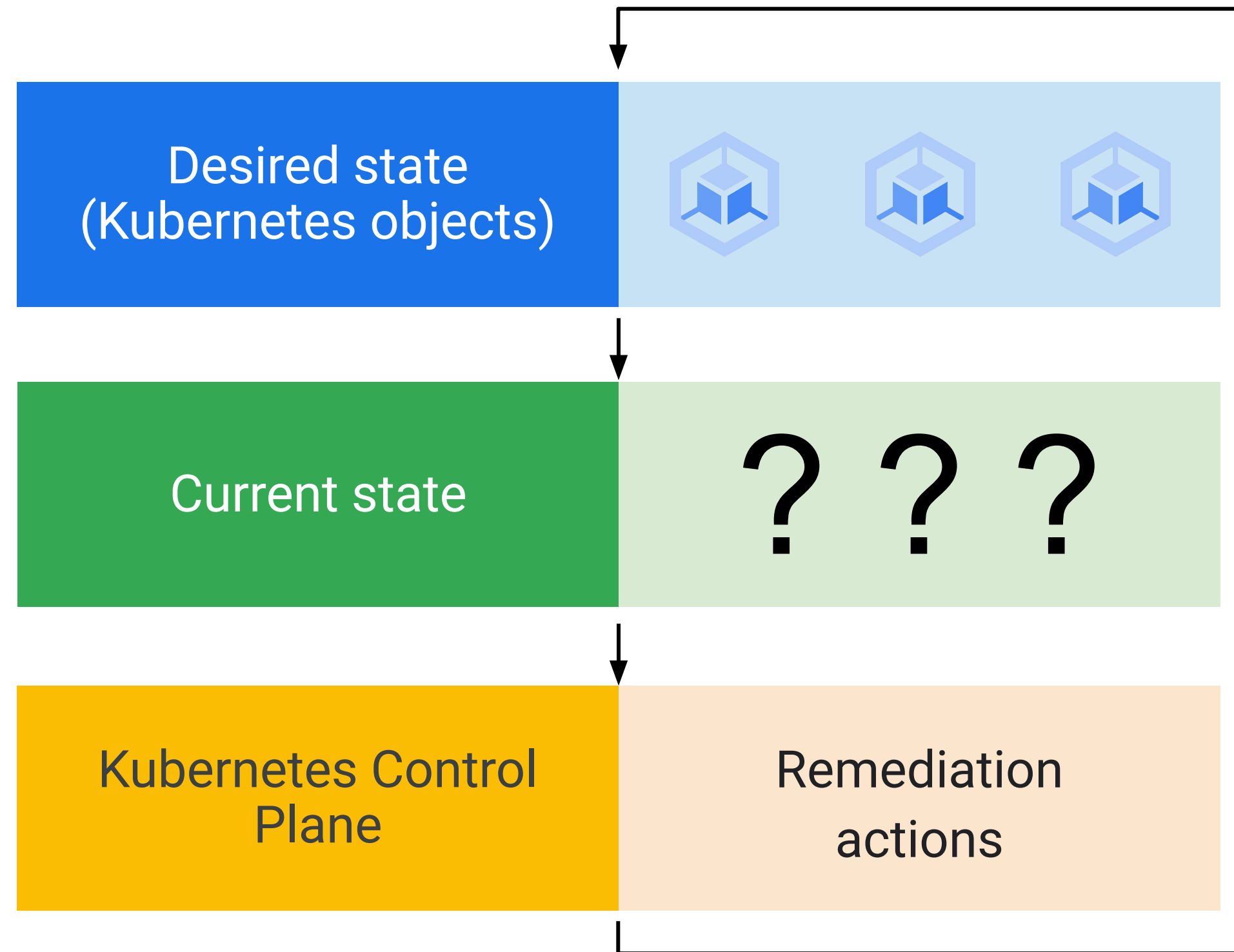
Running three nginx containers

You want three nginx containers running all the time

You declare objects that represent those containers

Kubernetes launches those objects and maintains them

Desired state compared to current state



Agenda

Kubernetes Concepts

Kubernetes Components

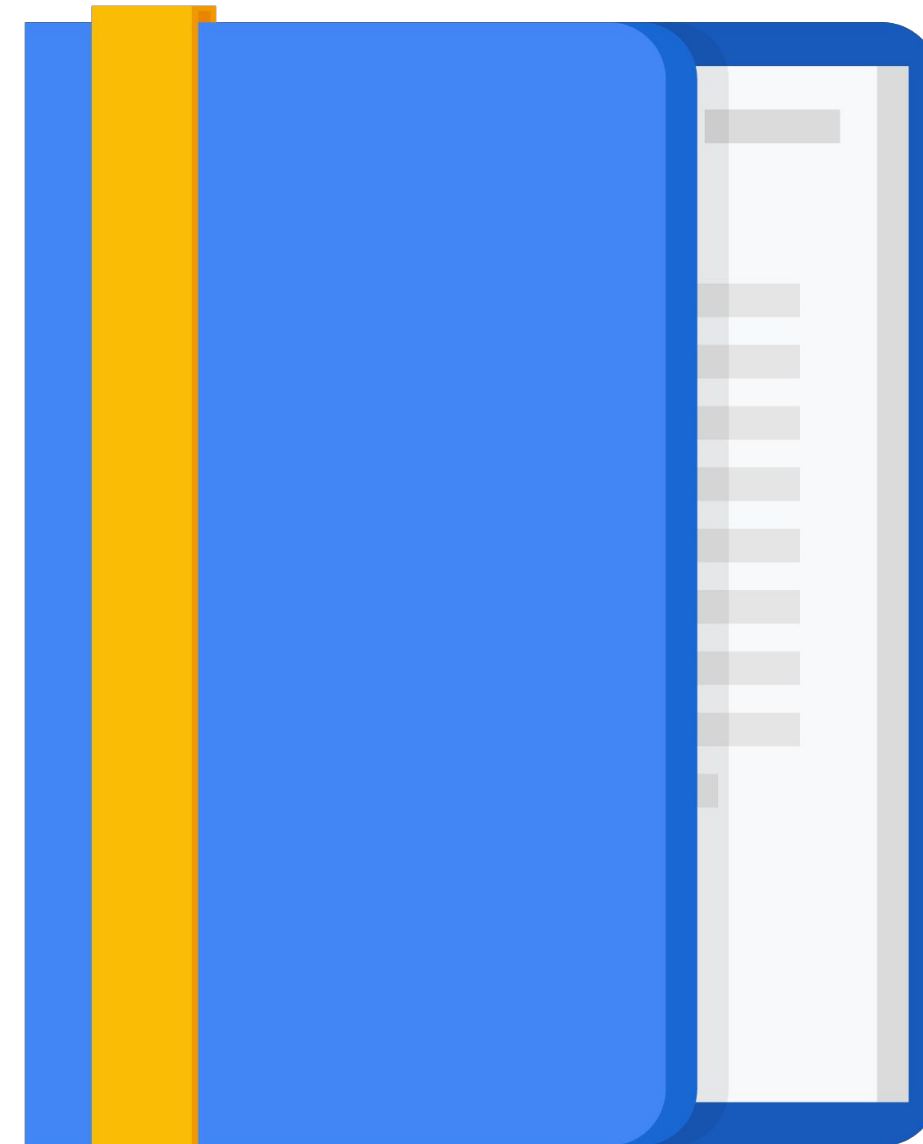
Google Kubernetes Engine Concepts

Object Management

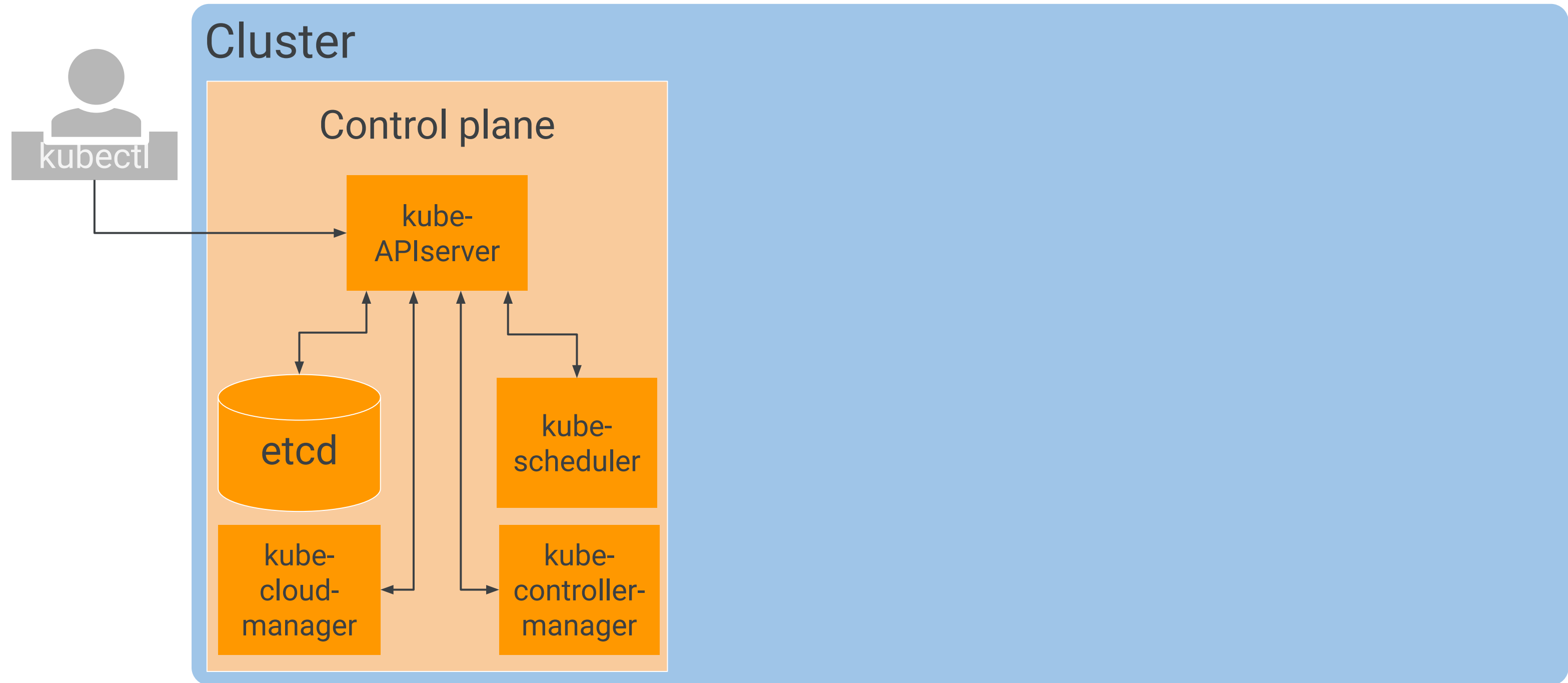
Lab: Deploying Google Kubernetes
Engine

Migrate for Anthos

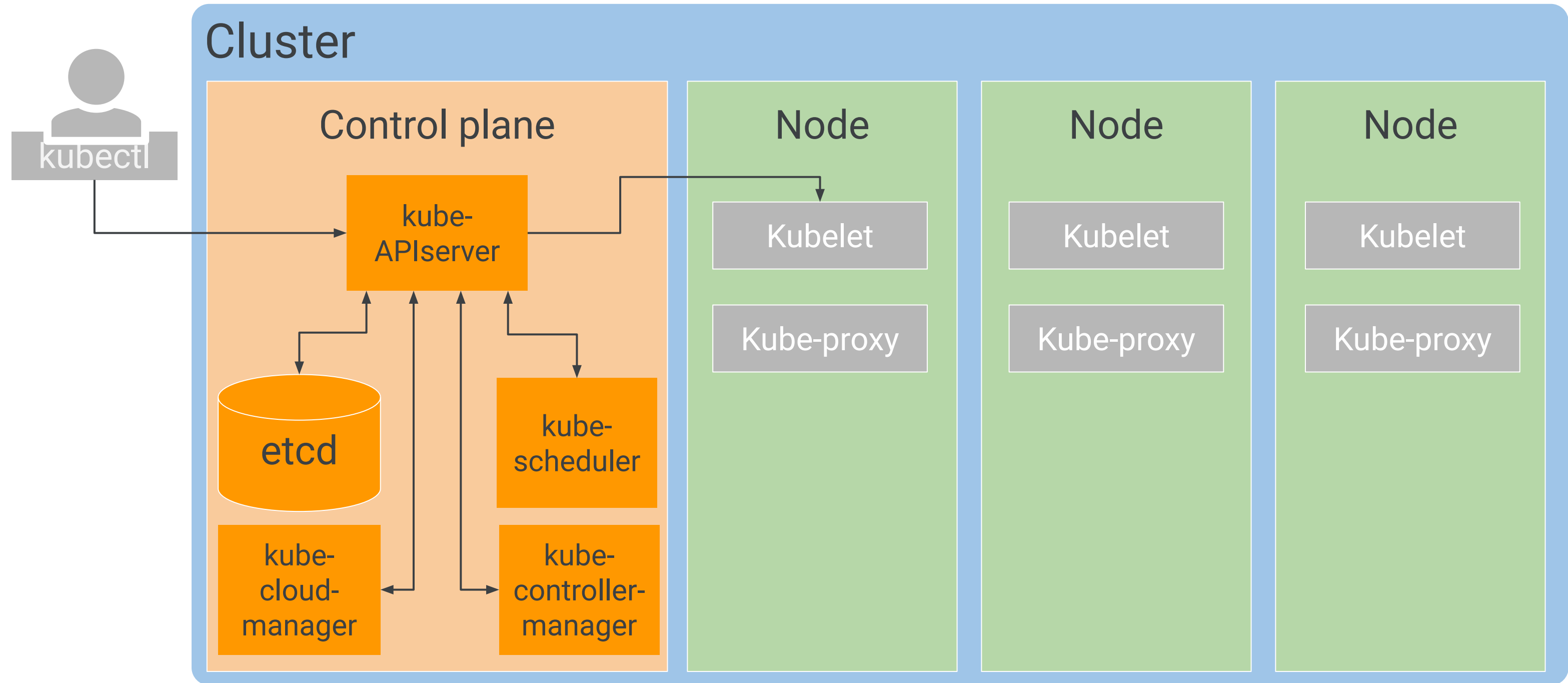
Summary



Cooperating processes make a Kubernetes cluster work



Cooperating processes make a Kubernetes cluster work



Agenda

Kubernetes Concepts

Kubernetes Components

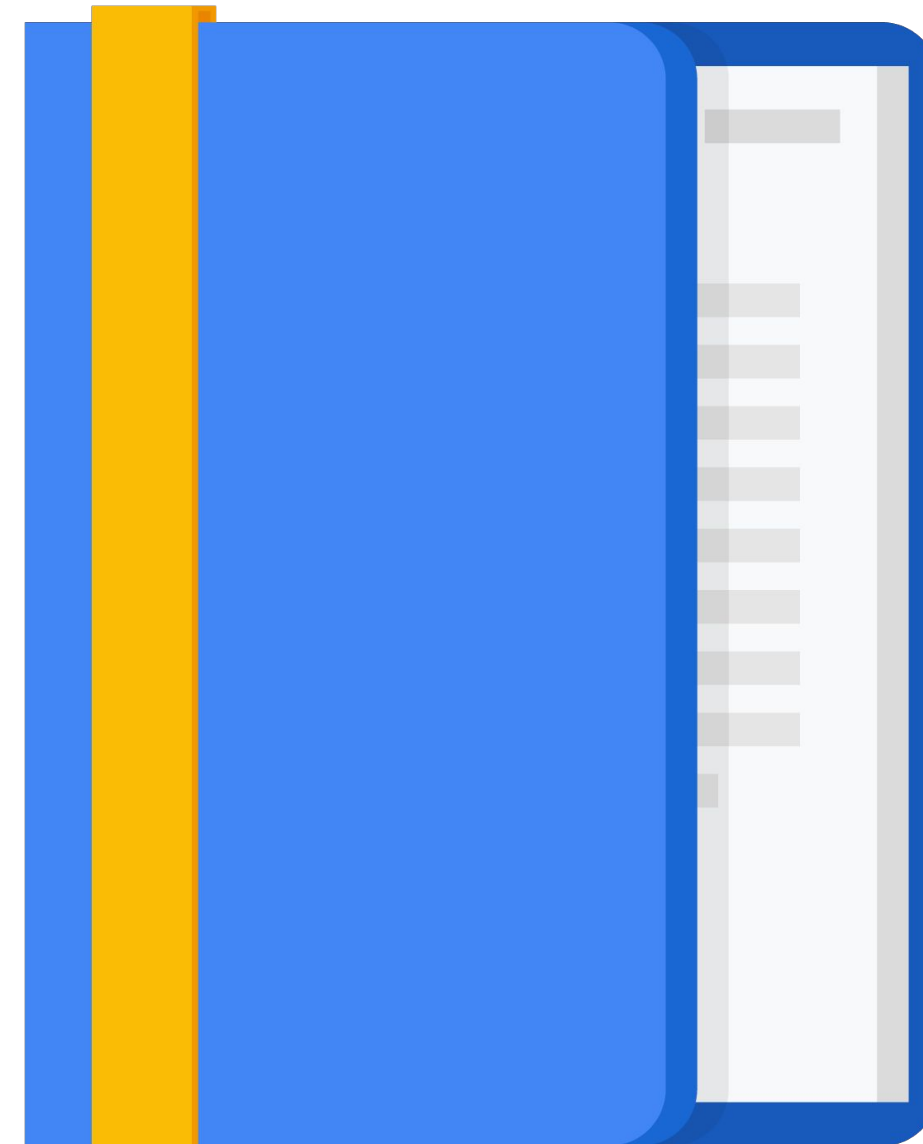
Google Kubernetes Engine Concepts

Object Management

Lab: Deploying Google Kubernetes
Engine

Migrate for Anthos

Summary



A comparison of the two GKE offerings

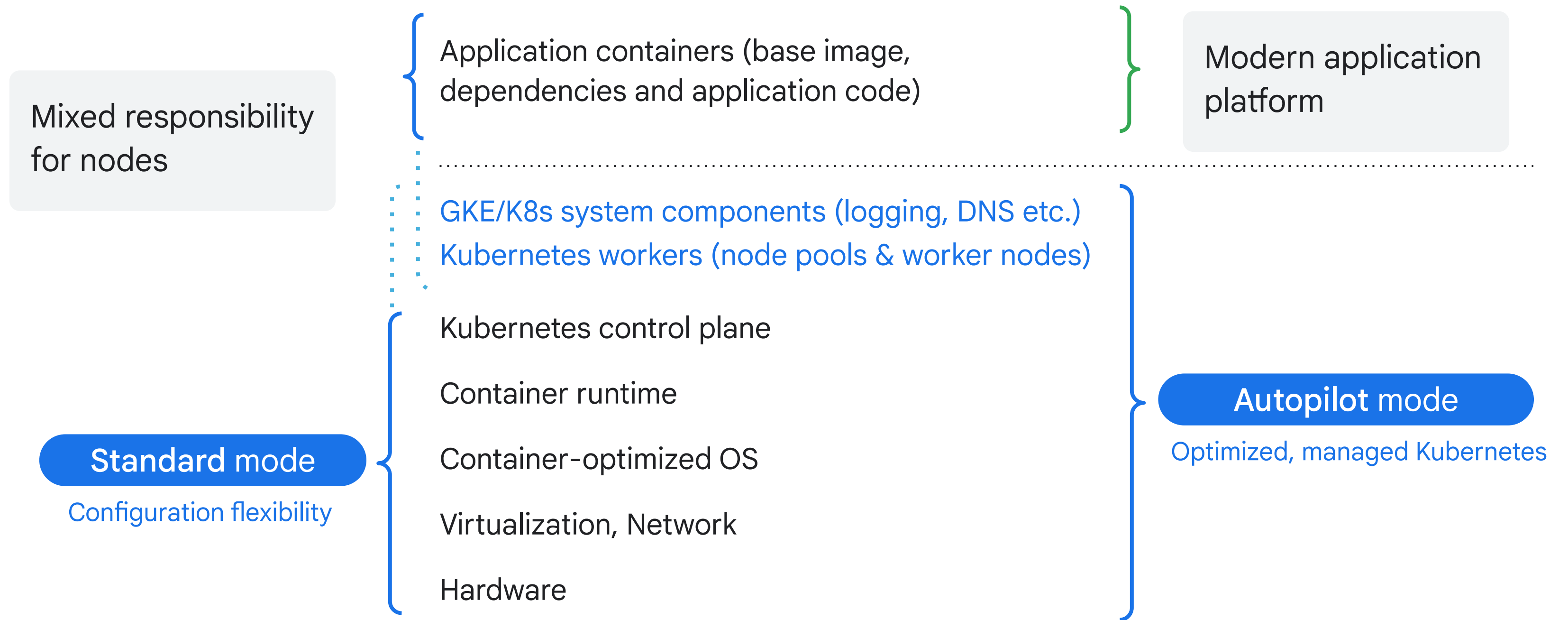
Autopilot mode

- Optimized Managed Kubernetes with hands-off experience
- Less management overhead, but less configuration abilities
- Only pay for what you use (Pod pricing)

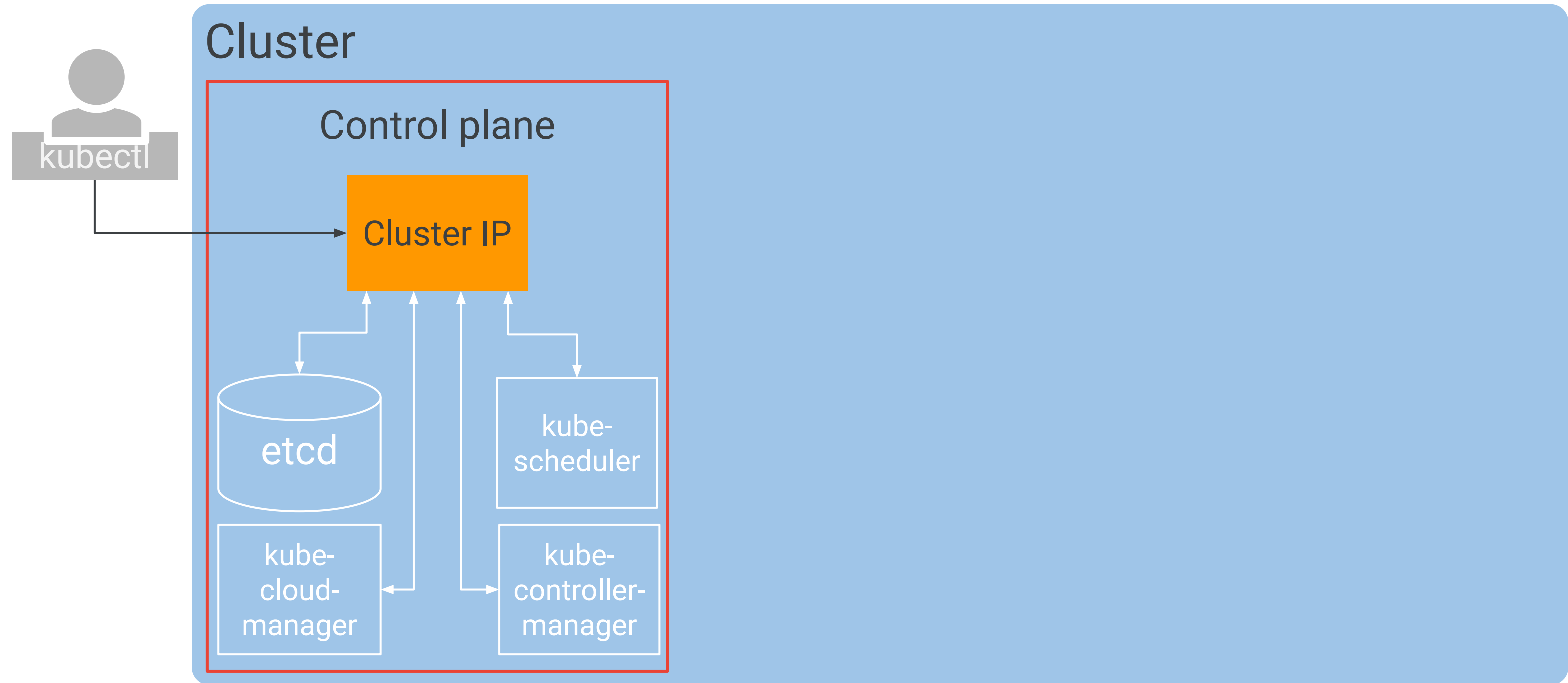
Standard mode

- Managed Kubernetes with configuration flexibility
- More management overhead but fine grained configuration.
- Pay for all provisioned infrastructure, regardless of utilization

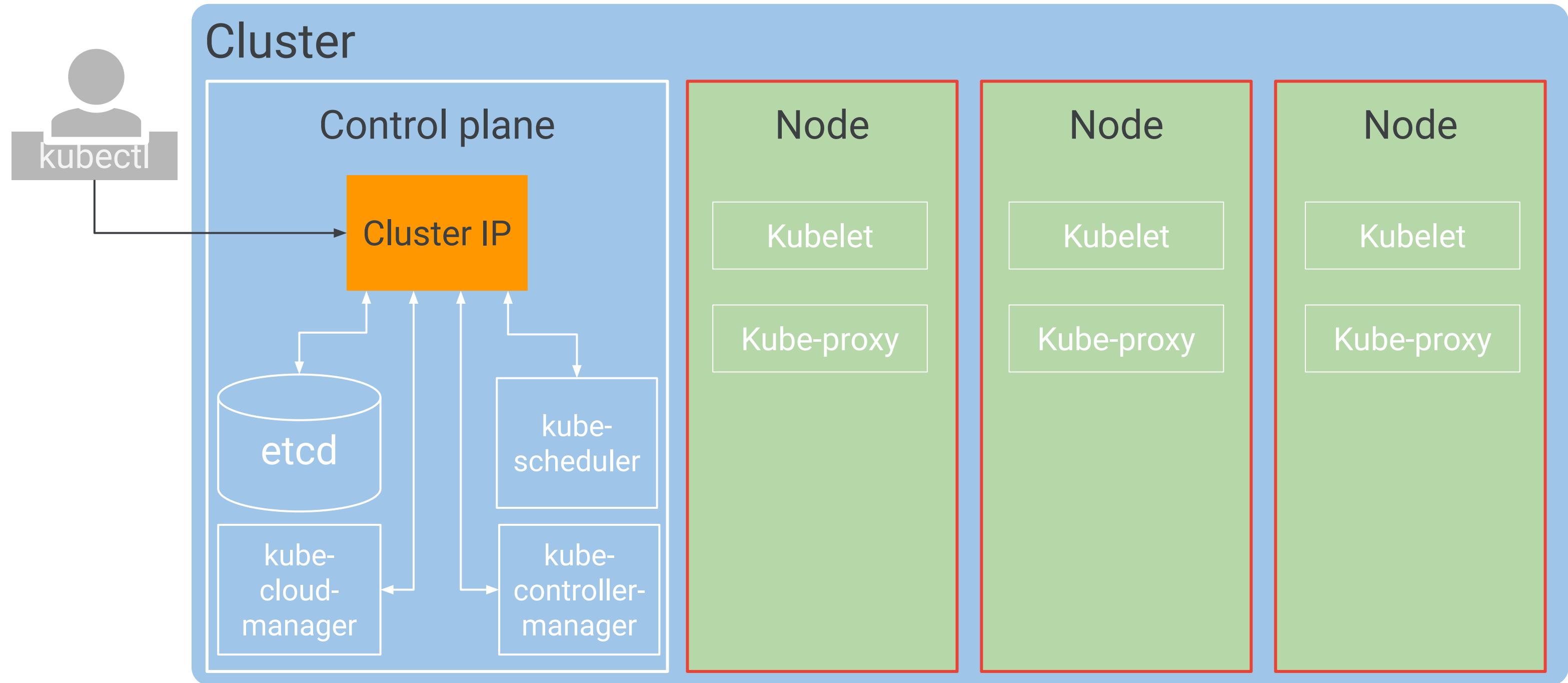
GKE manages the cluster infrastructure for you



GKE manages all the control plane components



GKE cluster nodes



Standard mode: You create and manage nodes in node pools.
Autopilot mode: Google manages nodes.

More about nodes for GKE Standard mode



Kubernetes doesn't create nodes.
Cluster admins create nodes and add them to Kubernetes.



GKE manages this by deploying and registering Compute Engine instances as nodes.

More about nodes for GKE Standard mode

Machine configuration

Machine family

GENERAL-PURPOSE COMPUTE-OPTIMIZED MEMORY-OPTIMIZED GPU

Machine types for common workloads, optimized for cost and flexibility

Series

E2

CPU platform selection based on availability

Machine type

e2-medium (2 vCPU, 4 GB memory)

	vCPU	Memory
	1 shared core	4 GB

CPU platform

Automatic

Machine configuration

Machine family

GENERAL-PURPOSE COMPUTE-OPTIMIZED MEMORY-OPTIMIZED GPU

High-performance machine types for compute-intensive workloads

Series

C2

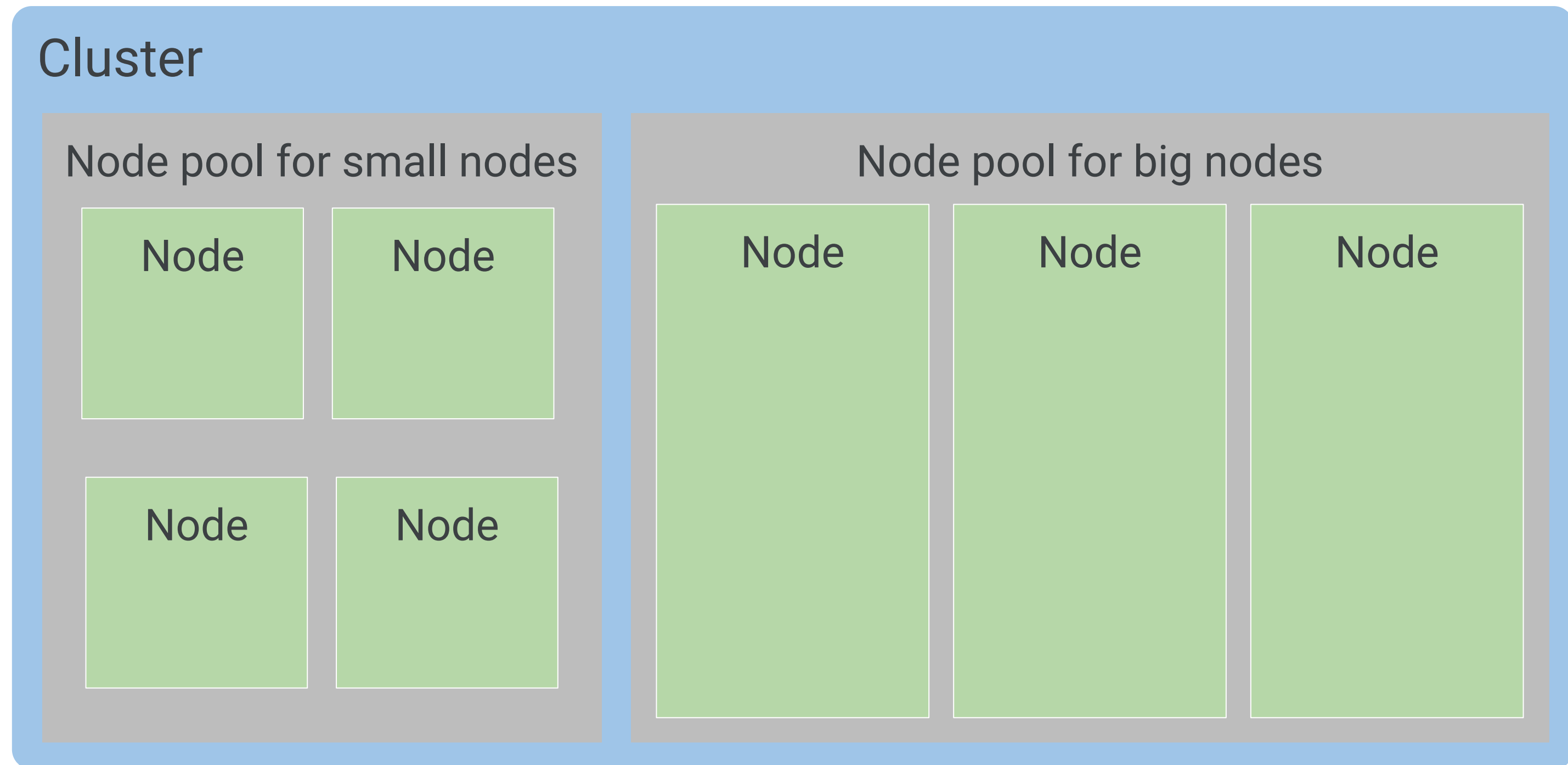
Powered by Intel Cascade Lake CPU platform

Machine type

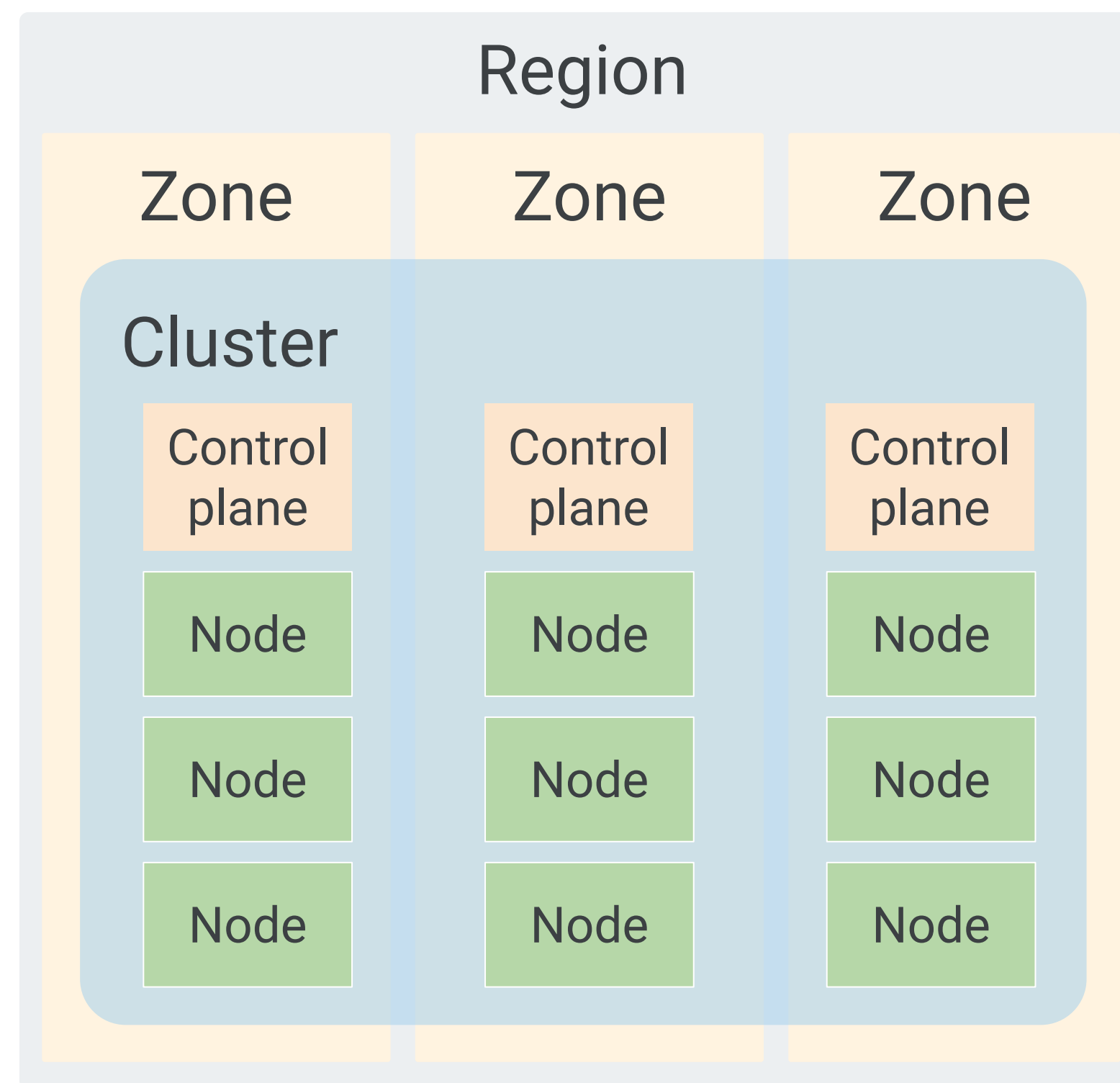
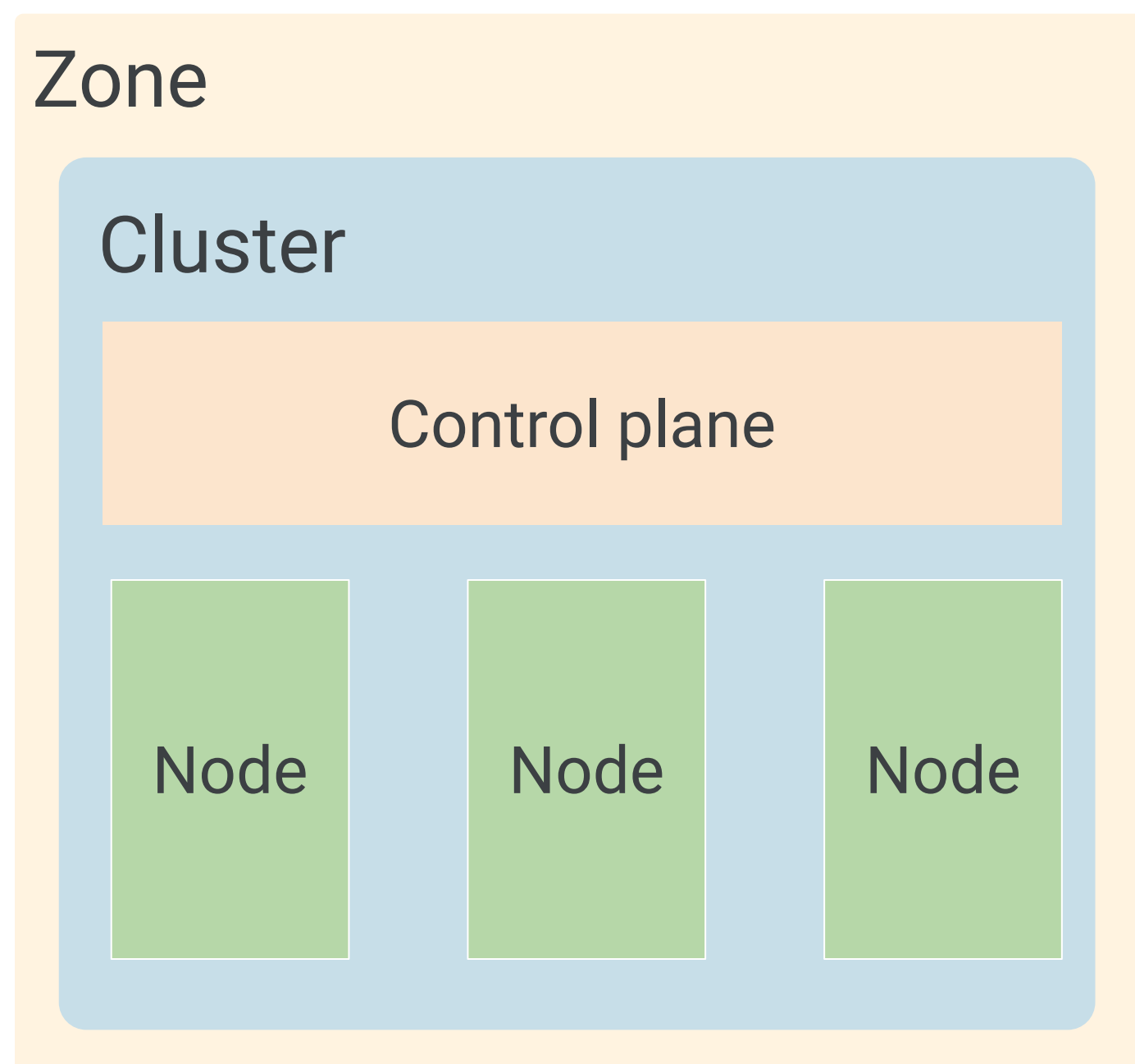
- c2-standard-4
4 vCPU, 16 GB memory
- c2-standard-8
8 vCPU, 32 GB memory
- c2-standard-16
16 vCPU, 64 GB memory
- c2-standard-30
30 vCPU, 120 GB memory
- c2-standard-60
60 vCPU, 240 GB memory

+ ADD GPU

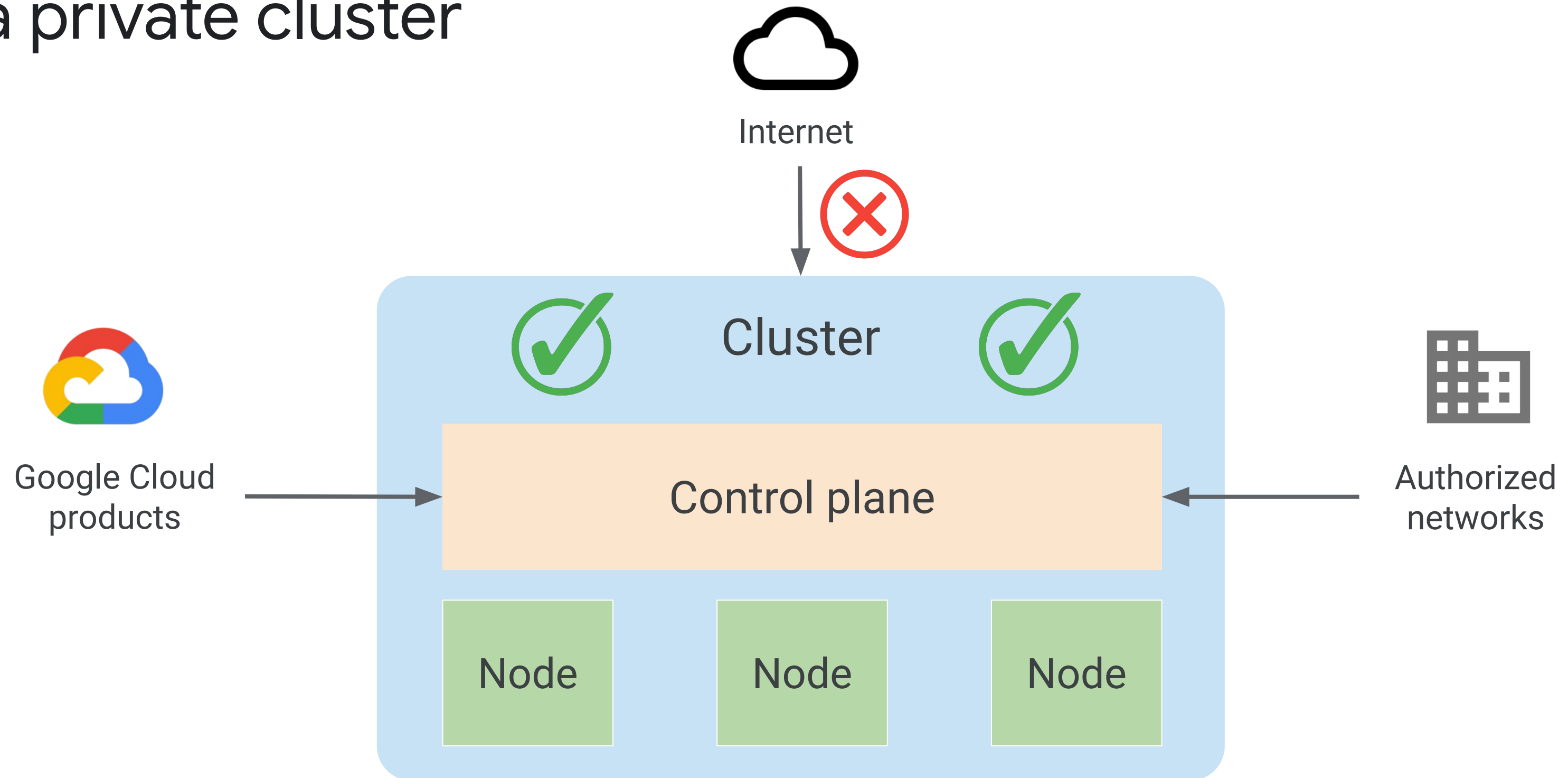
GKE Standard mode uses node pools to manage different kinds of nodes



Zonal versus regional clusters



A regional or zonal GKE cluster can also be set up as a private cluster



Agenda

Kubernetes Concepts

Kubernetes Components

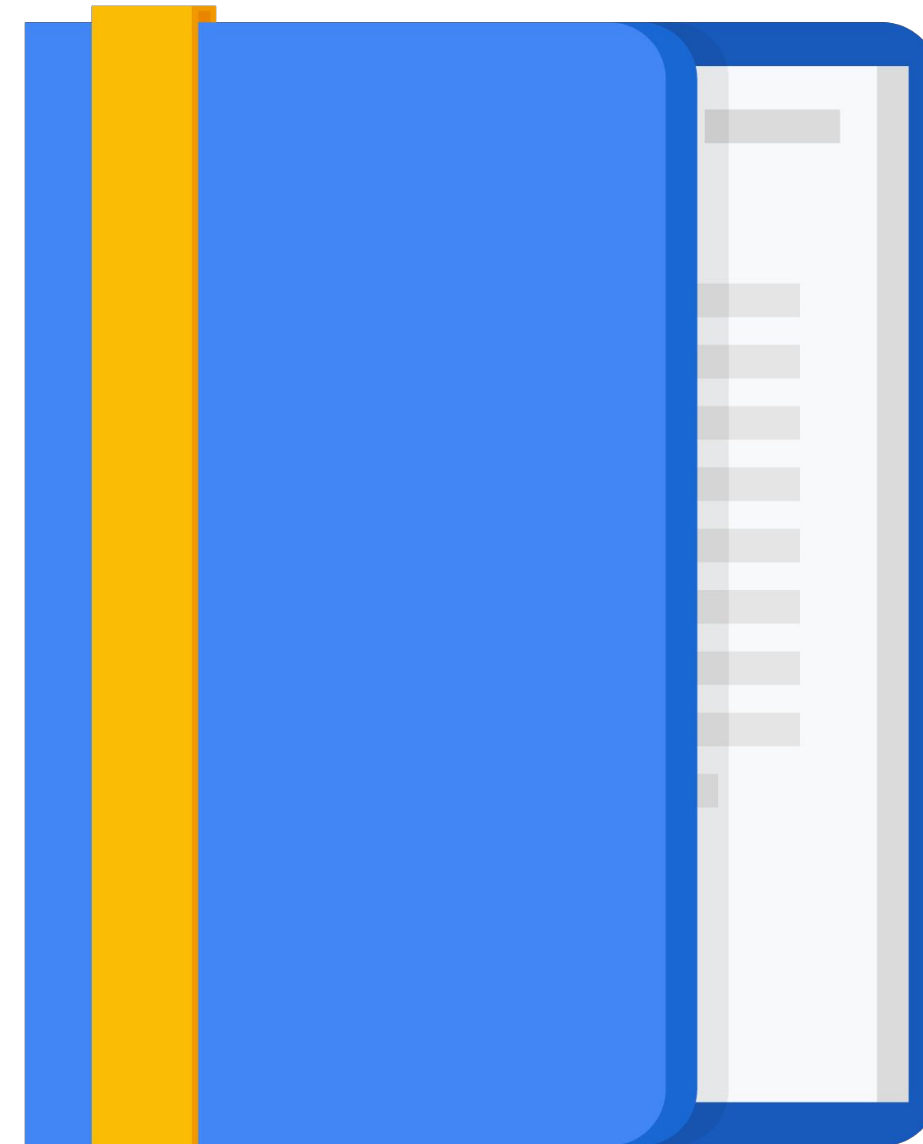
Google Kubernetes Engine Concepts

Object Management

Lab: Deploying Google Kubernetes
Engine

Migrate for Anthos

Summary

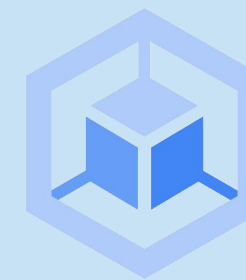
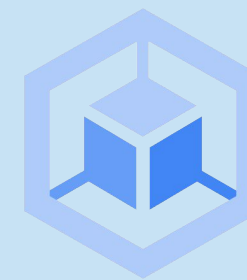
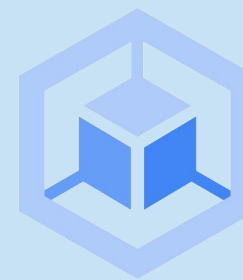


Running three nginx containers

You want three nginx containers running all the time

How do we create Pods for these containers?

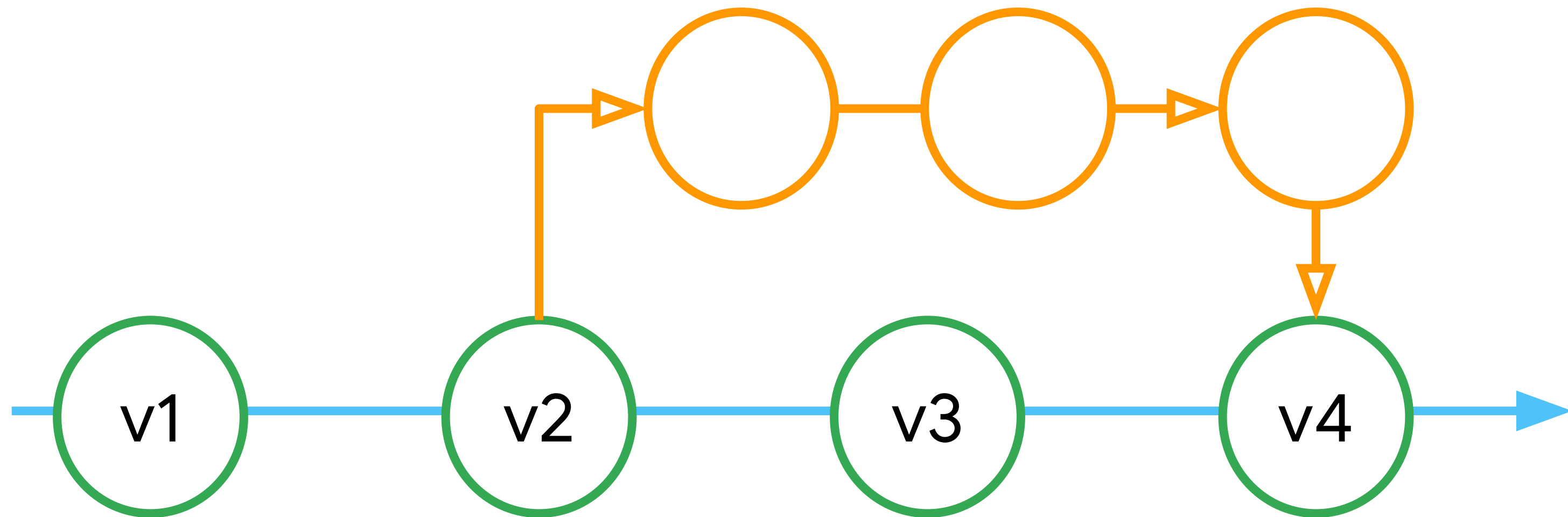
Kubernetes Objects
Object Spec
(Desired state)



Objects are defined in a YAML file

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    app: nginx
spec:
  containers:
  - name: nginx
    image: nginx:latest
```

Best practice tip: Use version control on YAML files



All objects are identified by a name

```
apiVersion: apps/v1
kind: Pod
metadata:
  name: nginx
[...]
```

Cannot have
two of the
same object
types with
same names

```
apiVersion: apps/v1
kind: Pod
metadata:
  name: nginx
[...]
```

```
apiVersion: apps/v1
kind: Pod
metadata:
  name: nginx
[...]
```

If an object is
deleted, the
name can be
reused

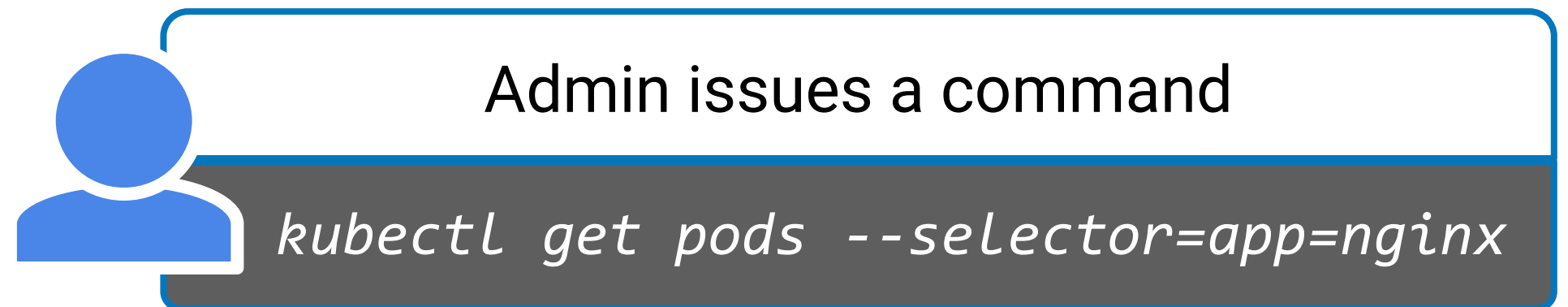
```
apiVersion: apps/v1
kind: Pod
metadata:
  name: nginx
[...]
```

All objects are assigned a unique identifier (UID) by Kubernetes

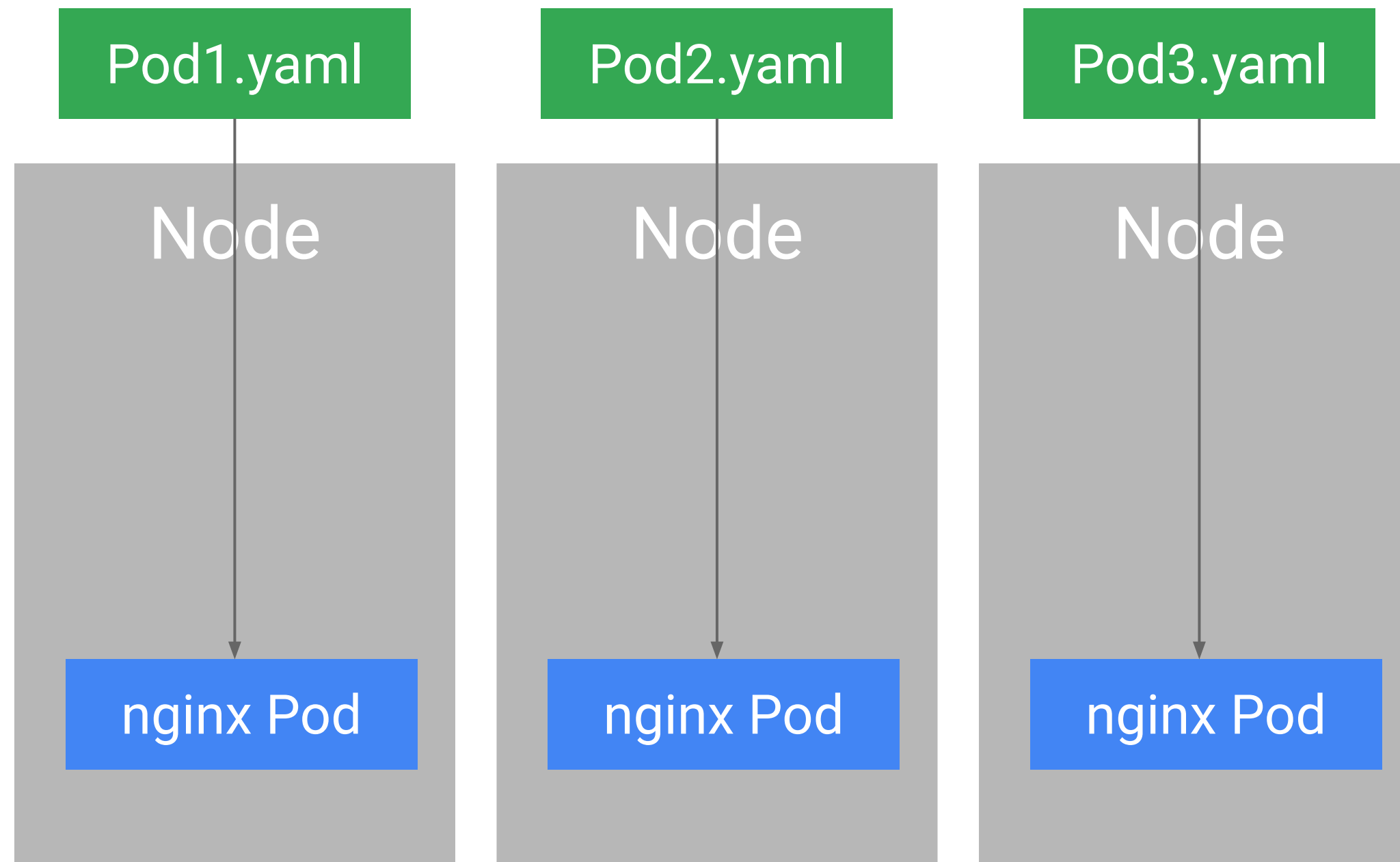
```
apiVersion: apps/v1
kind: Pod
metadata:
  name: nginx
  uid: 4dd474fn-f389-11f8-b38c-42010a8009z7
[...]
```

Labels can be matched by label selectors

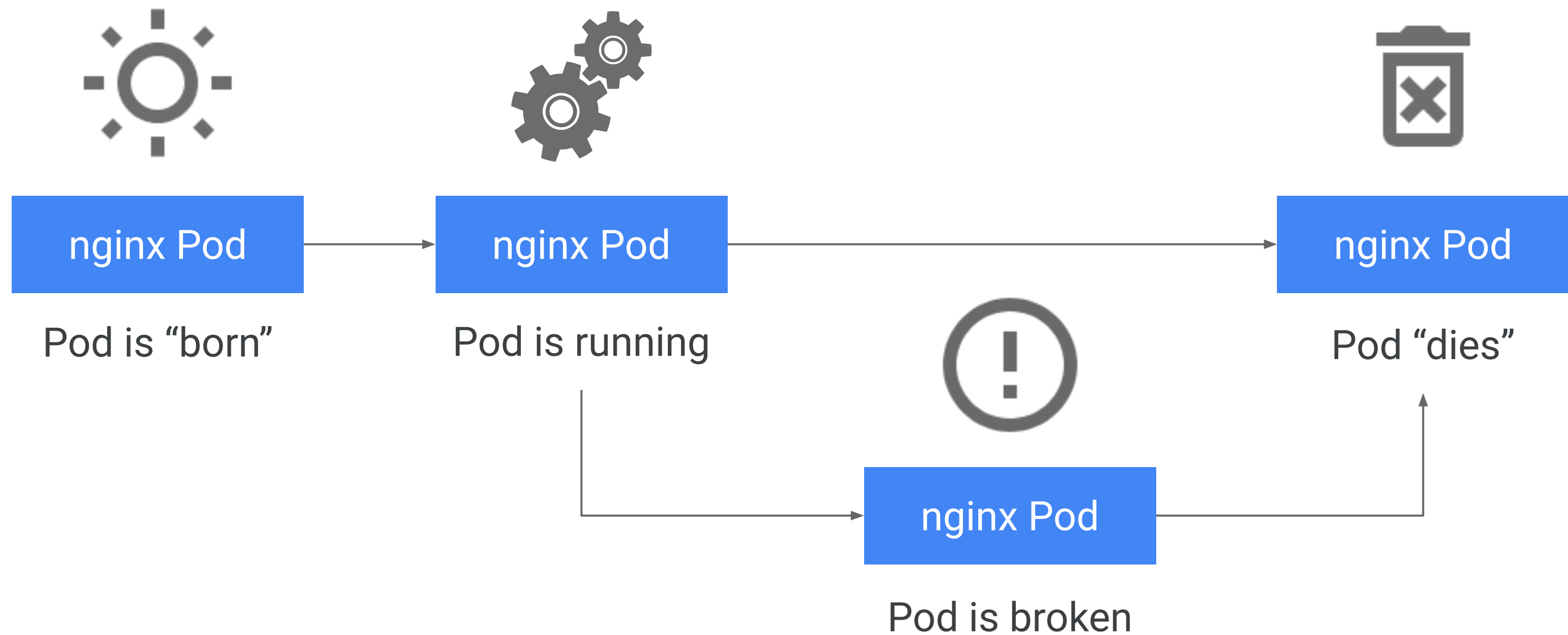
```
apiVersion: apps/v1
kind: Pod
metadata:
  name: nginx
  labels:
    app: nginx
    env: dev
    stack: frontend
spec:
  selector:
    matchLabels:
      app: nginx
```



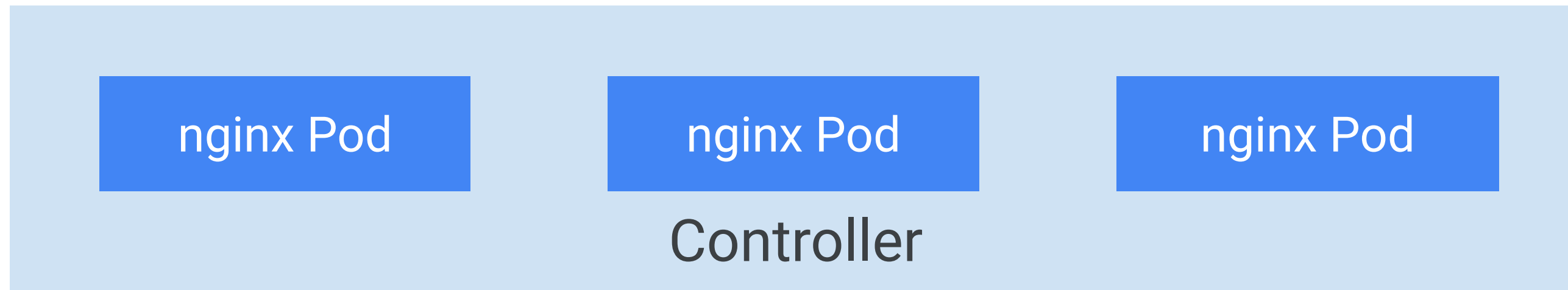
A workload is spread evenly across available nodes by default



Pods have a life cycle



Pods and Controller Objects



Controller object
types

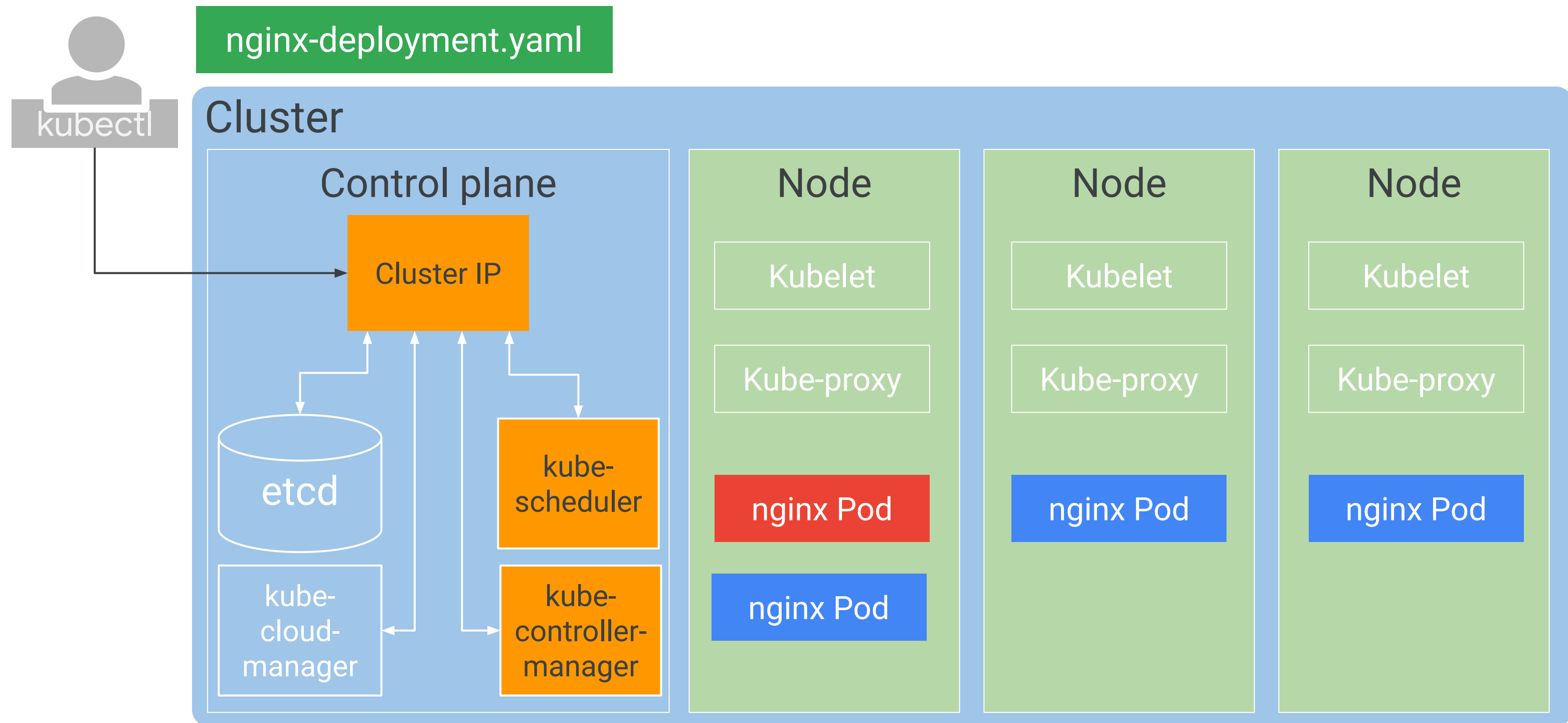
- Deployment
- StatefulSet
- DaemonSet
- Job

Deployments are a great choice for long-lived software components

You want three nginx containers running all the time

How does Kubernetes maintain 3 nginx containers at any given time?

A Deployment maintains the desired state



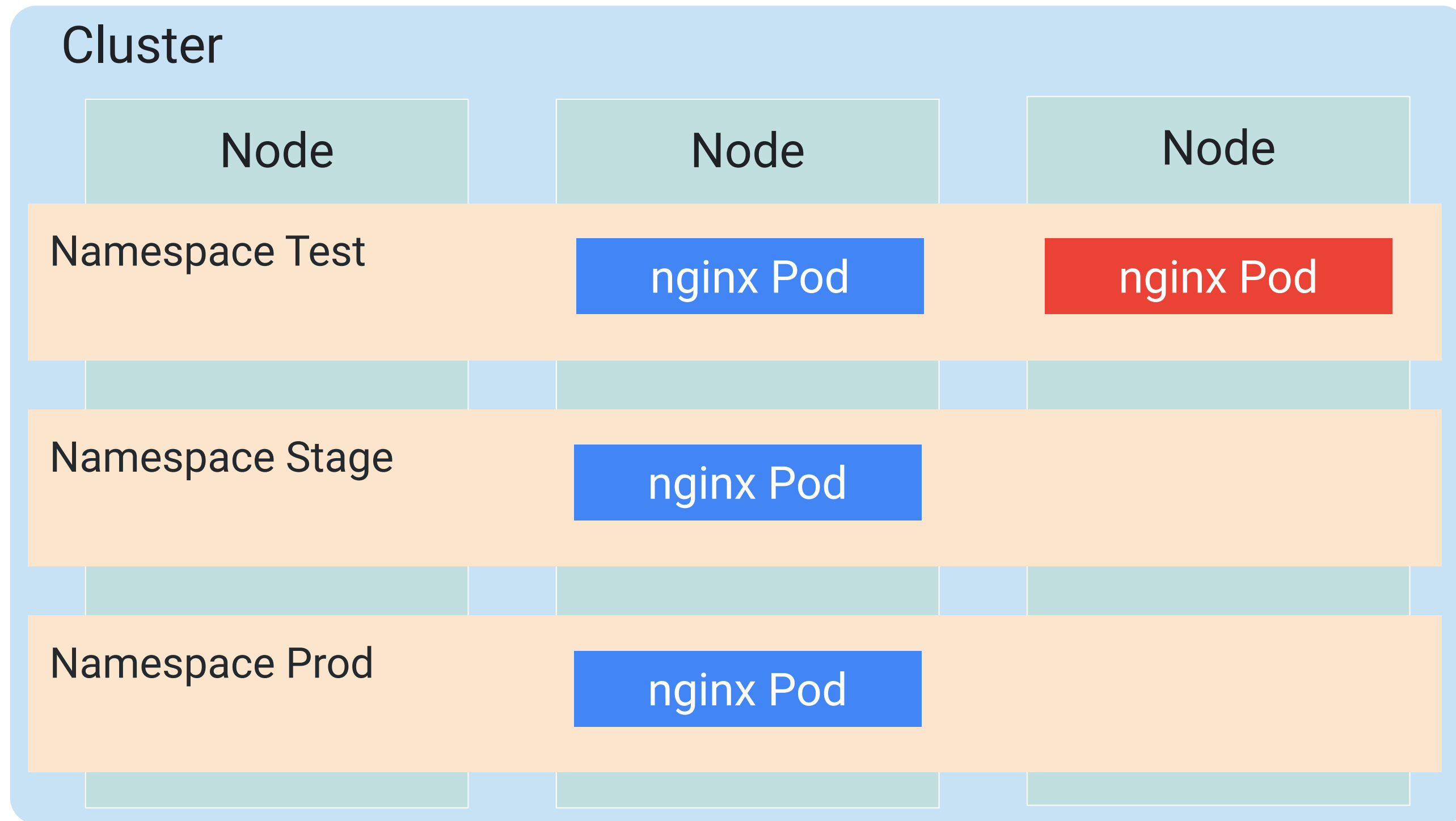
Deployments ensure that sets of Pods are running

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:latest
```

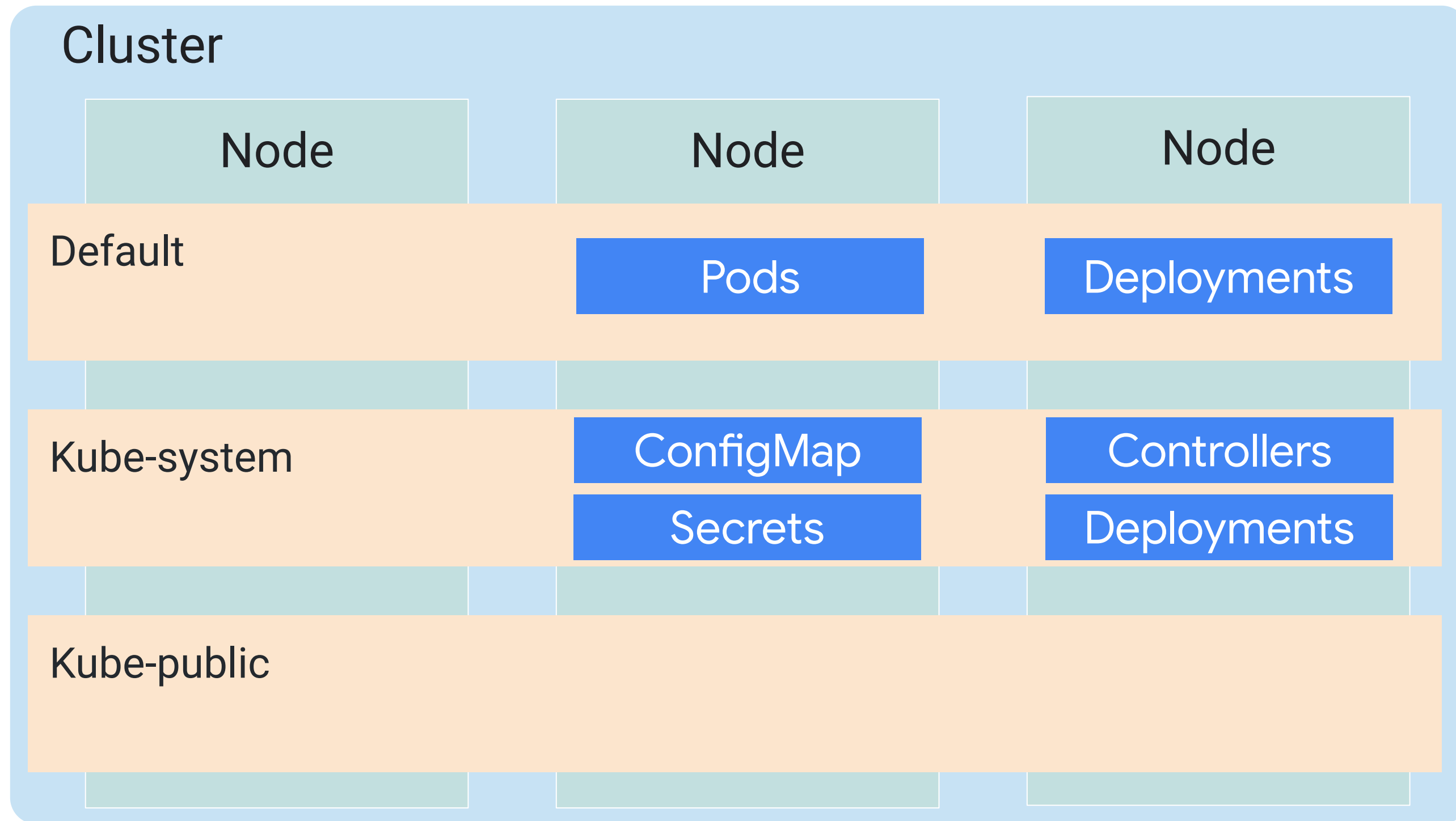
Resource management for Pods and Containers

- ✓ Important that containers have enough resources to run.
- ✓ Applications could use more resources than they should.
- ✓ CPU and memory (RAM) resources are the most common resources specified.

Namespaces provide scope for naming resources



There are three initial namespaces in a cluster



Best practice tip: namespace-neutral YAML



Most flexible:

```
kubectl -n demo apply -f mypod.yaml
```



Legal but less flexible:

```
apiVersion: v1
kind: Pod
metadata:
  name: mypod
  namespaces: demo
```

Agenda

Kubernetes Concepts

Kubernetes Components

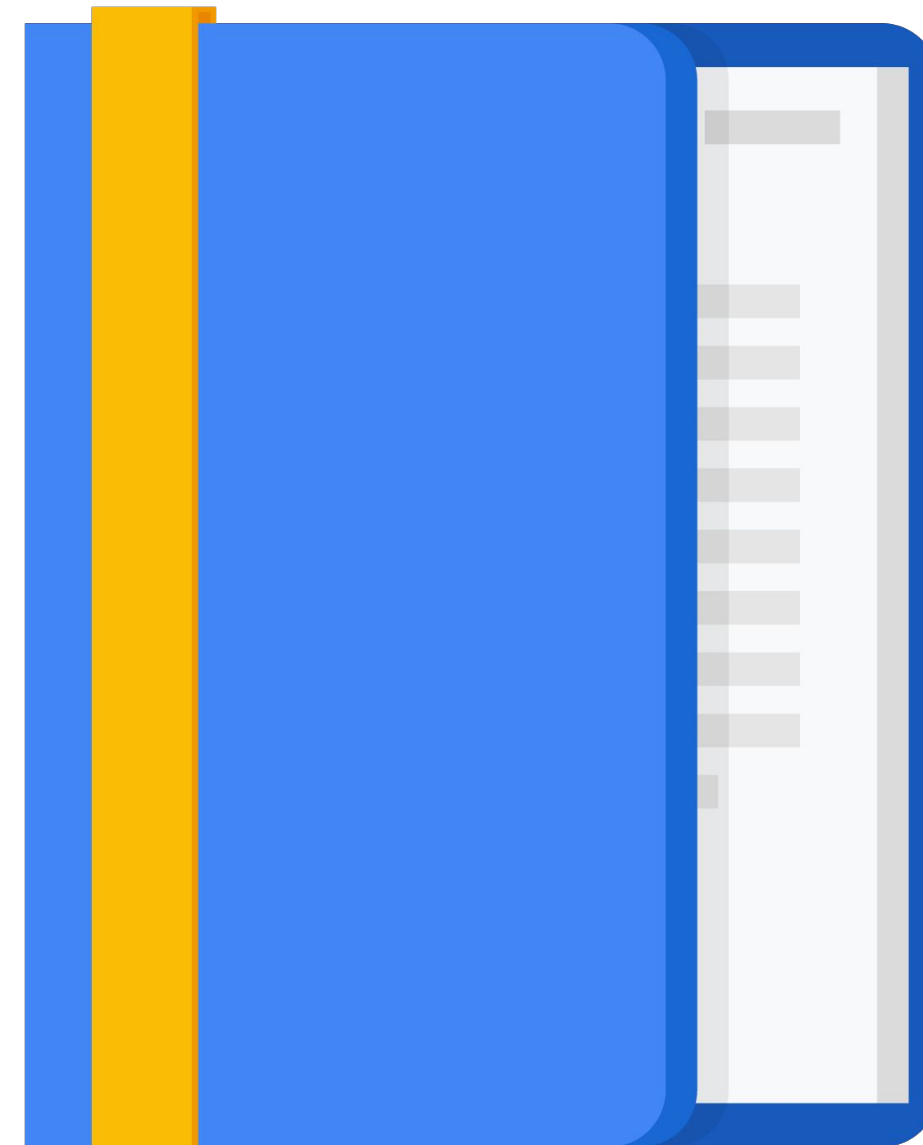
Google Kubernetes Engine Concepts

Object Management

Lab: Deploying Google Kubernetes
Engine

Migrate for Anthos

Summary



Lab Intro

Deploying Google Kubernetes
Engine



Agenda

Kubernetes Concepts

Kubernetes Components

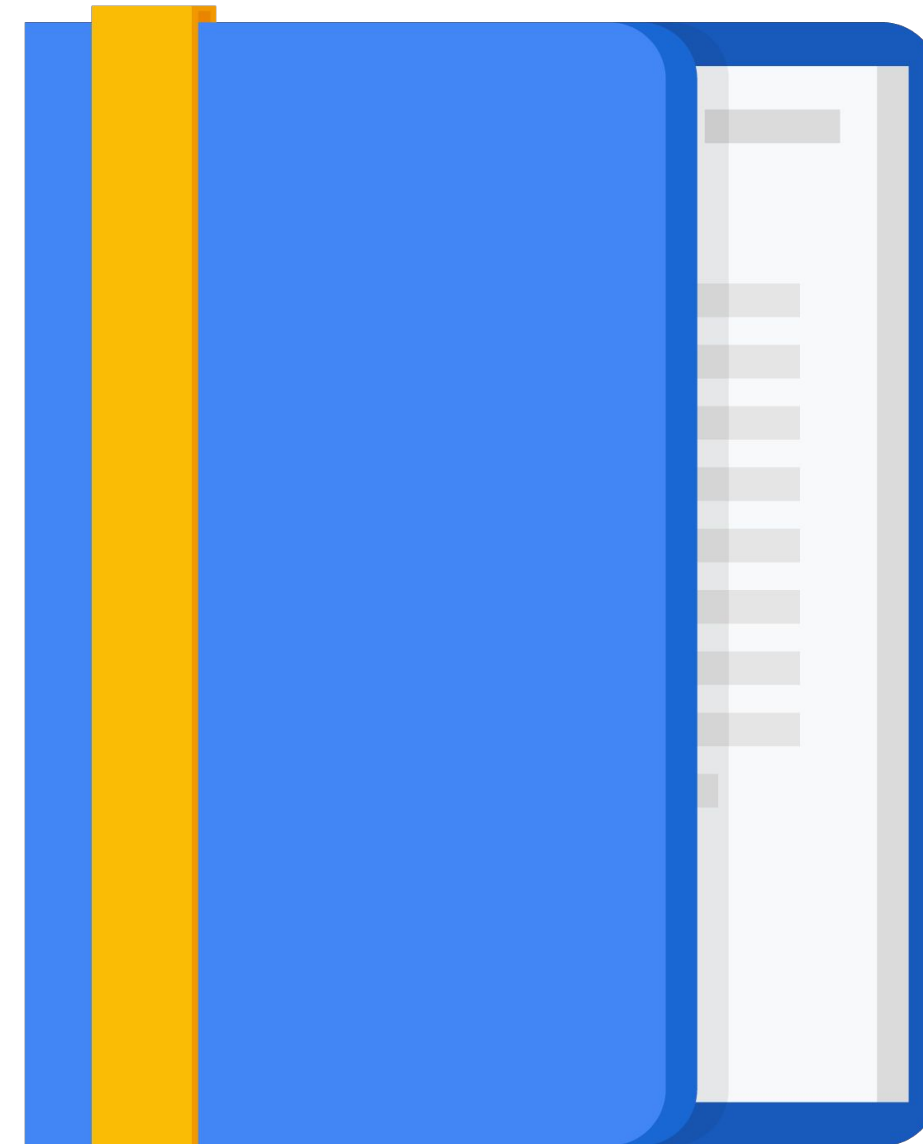
Google Kubernetes Engine Concepts

Object Management

Lab: Deploying Google Kubernetes
Engine

Migrate for Anthos

Summary



Migrate for Anthos moves VMs to containers



Move and convert workloads into containers.



Workloads can start as physical servers or VMs.



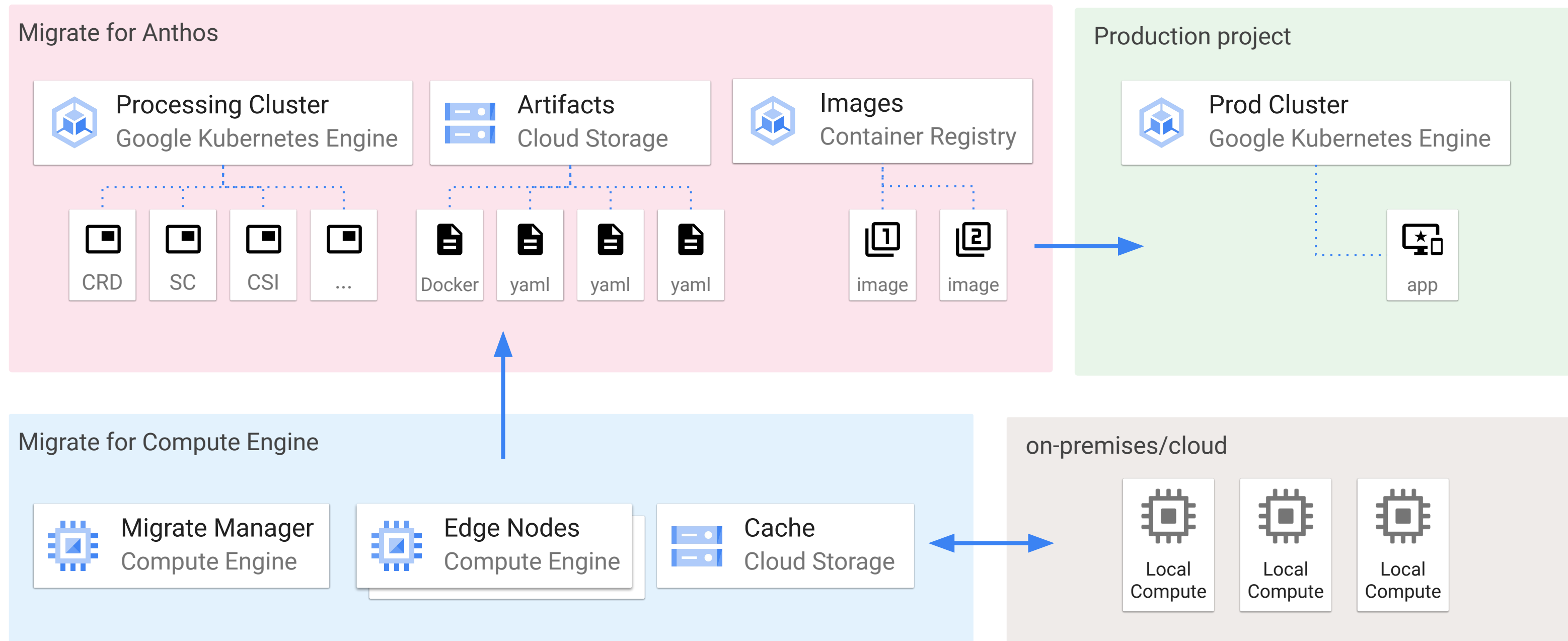
Moves workload compute to container immediately (<10 min).



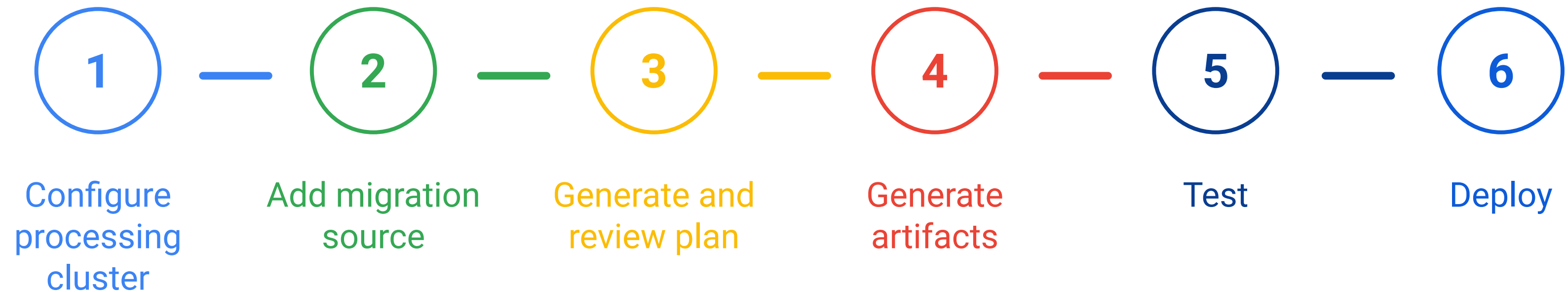
Data can be migrated all at once or "streamed" to the cloud until the app is live in the cloud.



A migration requires an architecture to be built



A migration is a multi-step process



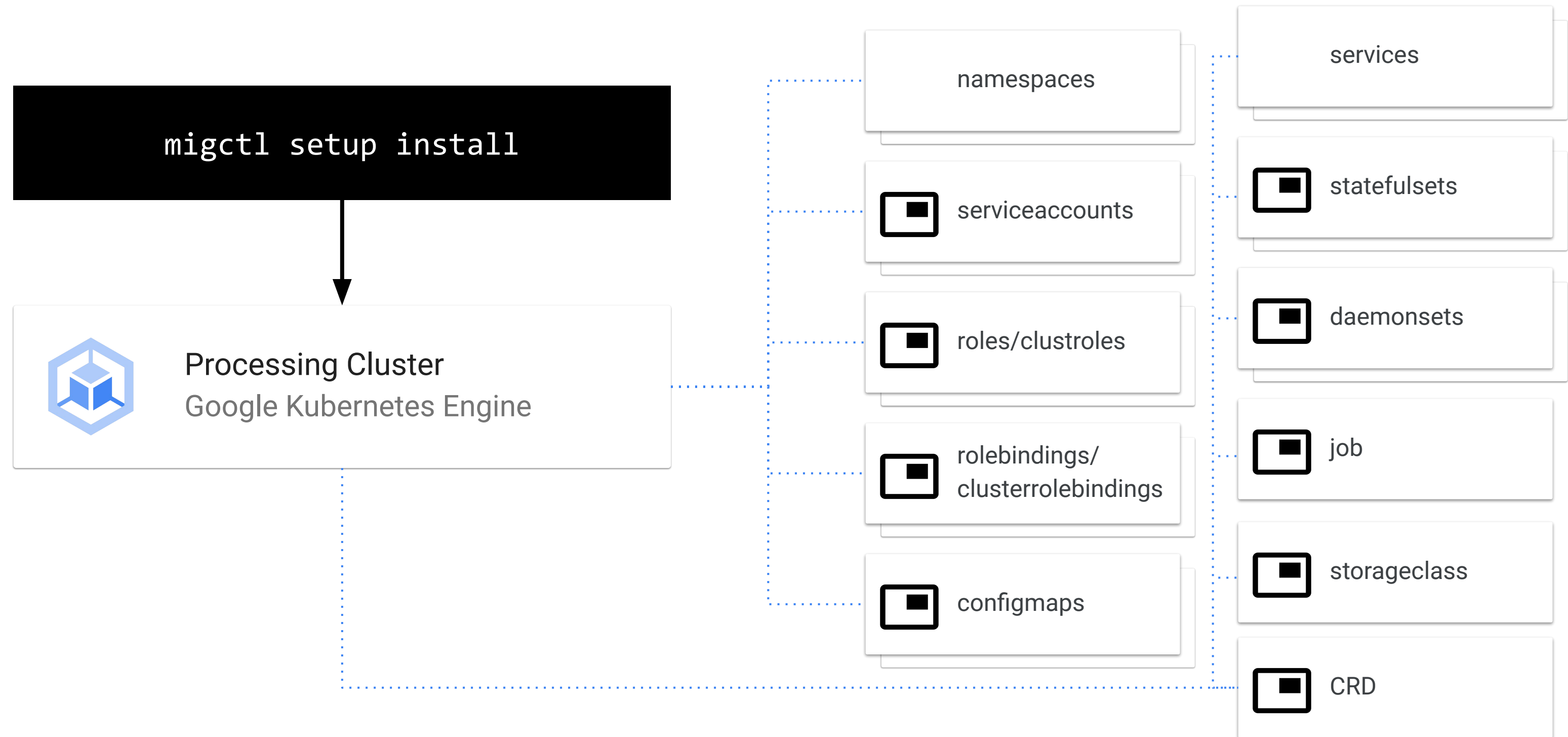
Migrate for Anthos requires a processing cluster

```
gcloud container --project $PROJECT_ID \
clusters create $CLUSTER_NAME \
--zone $CLUSTER_ZONE \
--username "admin" \
--cluster-version 1.14 \
--machine-type "n1-standard-4" \
--image-type "UBUNTU" \
--num-nodes 1 \
--enable-stackdriver-kubernetes \
--scopes "cloud-platform" \
--enable-ip-alias \
--tags="http-server"
```



Processing Cluster
Google Kubernetes Engine

Installing Migrate for Anthos uses `migctl`

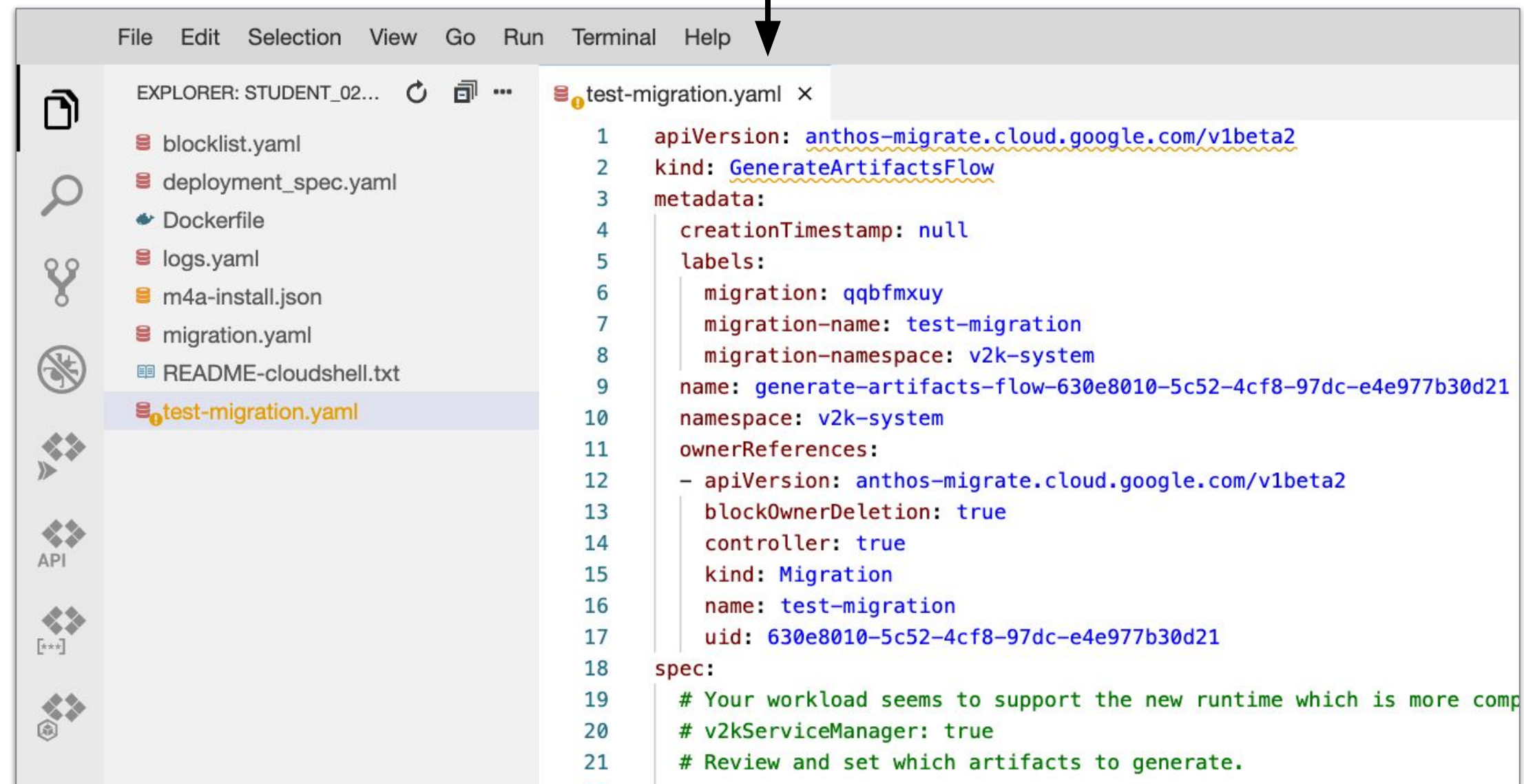


Adding a source enables migrations from a specific environment

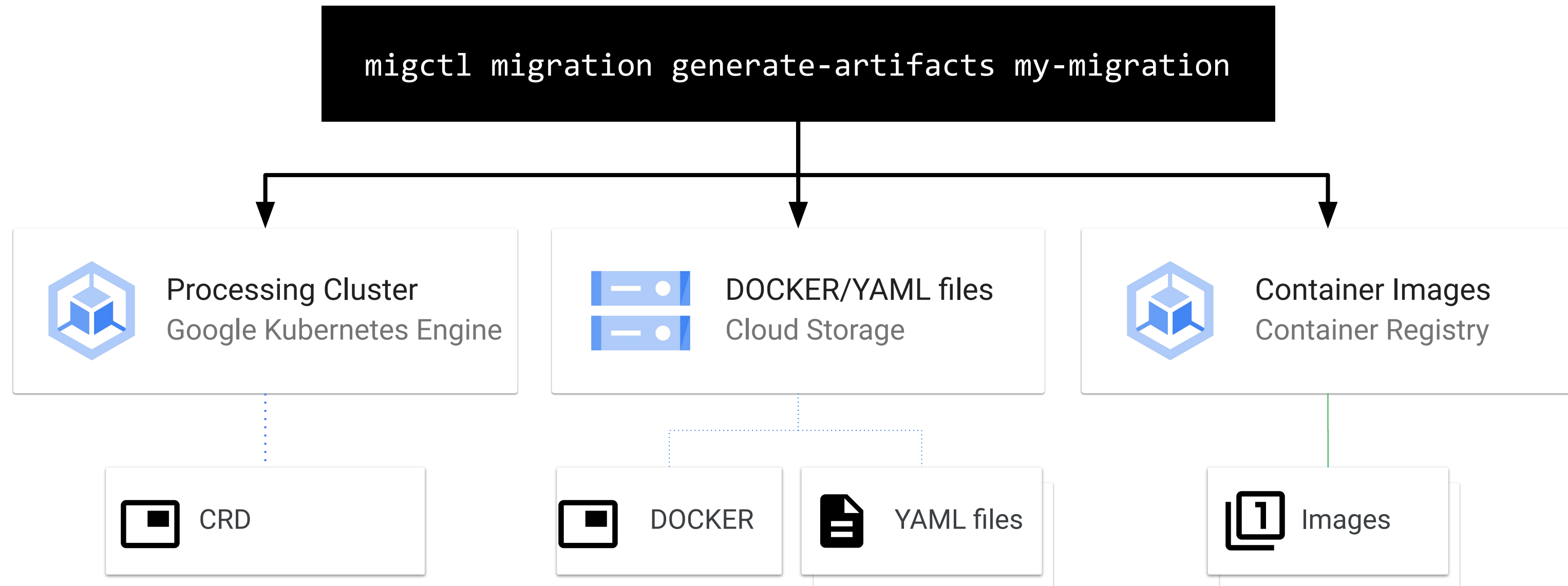
```
migctl source create ce my-ce-src --project my-project --zone zone
```

Creating a migration generates a migration plan

```
migctl migration create test-migration --source my-ce-src --vm-id my-id --intent Image
```

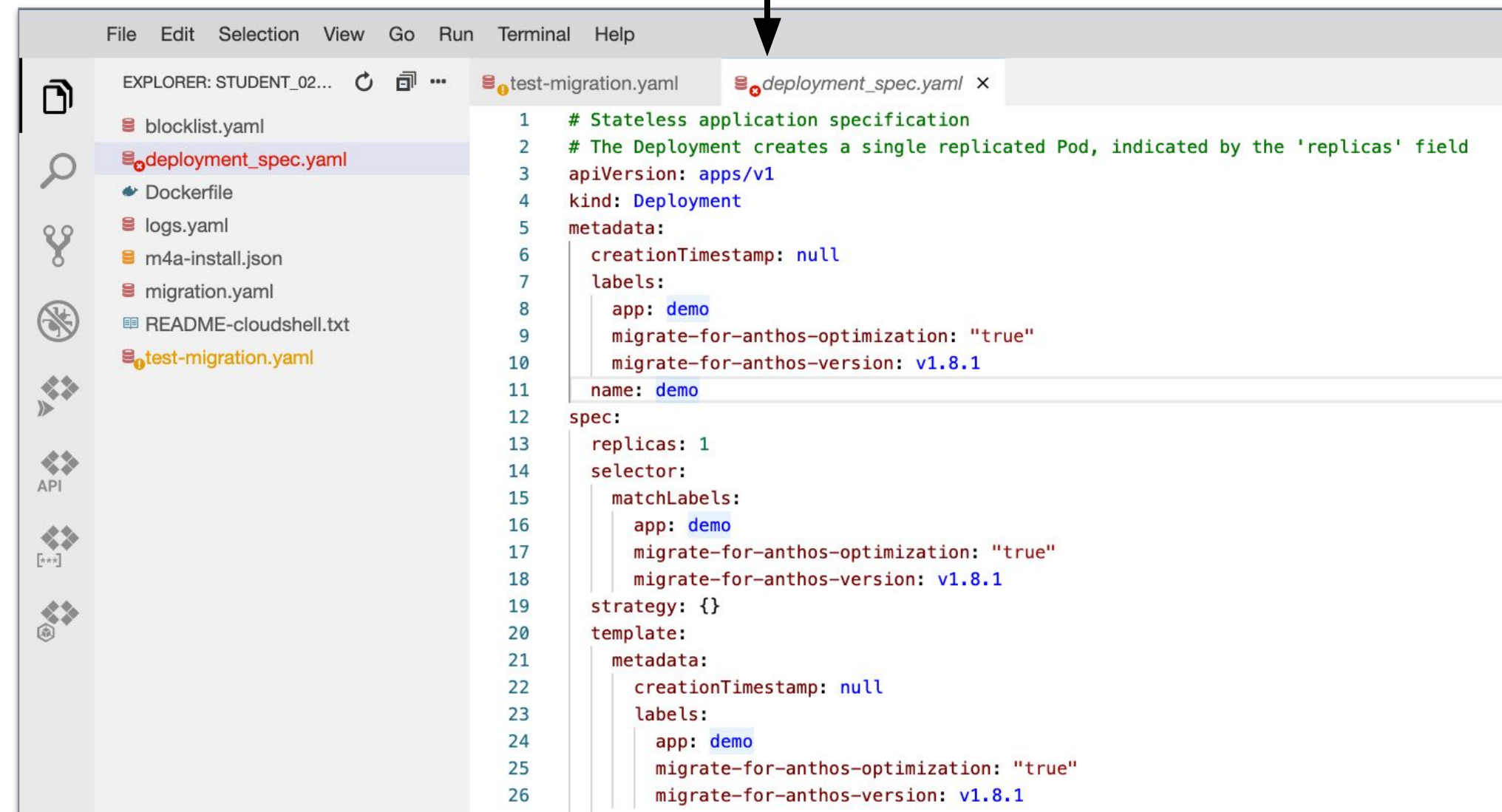


Executing a migration generates resources and artifacts



Deployment files typically need modification

```
migctl migration get-artifacts test-migration
```



```
File Edit Selection View Go Run Terminal Help
EXPLORER: STUDENT_02... test-migration.yaml deployment_spec.yaml x
  blocklist.yaml
  deployment_spec.yaml
  Dockerfile
  logs.yaml
  m4a-install.json
  migration.yaml
  README-cloudshell.txt
  test-migration.yaml
API
[+]

1  # Stateless application specification
2  # The Deployment creates a single replicated Pod, indicated by the 'replicas' field
3  apiVersion: apps/v1
4  kind: Deployment
5  metadata:
6    creationTimestamp: null
7    labels:
8      app: demo
9      migrate-for-anthos-optimization: "true"
10     migrate-for-anthos-version: v1.8.1
11   name: demo
12 spec:
13   replicas: 1
14   selector:
15     matchLabels:
16       app: demo
17       migrate-for-anthos-optimization: "true"
18       migrate-for-anthos-version: v1.8.1
19   strategy: {}
20   template:
21     metadata:
22       creationTimestamp: null
23     labels:
24       app: demo
25       migrate-for-anthos-optimization: "true"
26       migrate-for-anthos-version: v1.8.1
```

Apply the configuration to deploy the workload

```
kubectl apply -f deployment_spec.yaml
```

Agenda

Kubernetes Concepts

Kubernetes Components

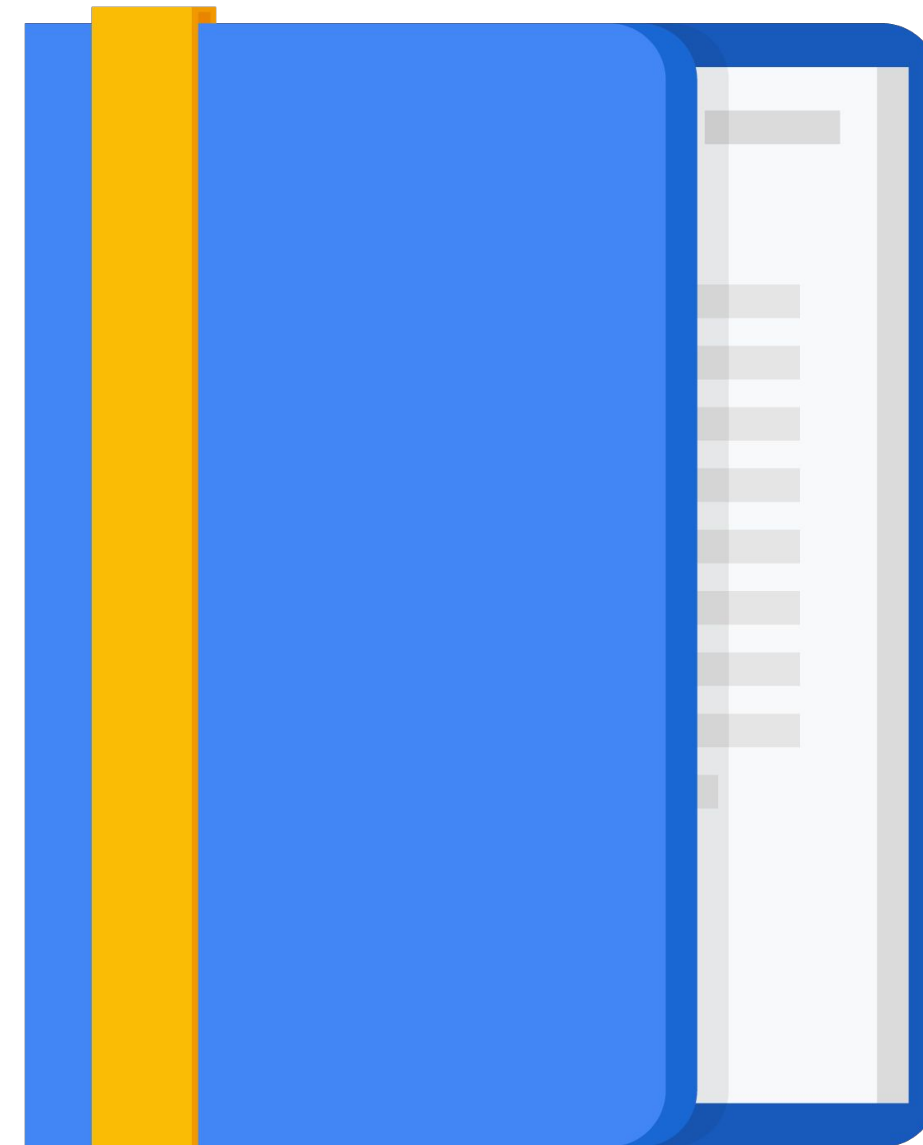
Google Kubernetes Engine
Concepts

Object Management

Lab: Deploying Google Kubernetes
Engine

Migrate for Anthos

Summary



Summary

Kubernetes controllers keep the cluster state matching the desired state.

Kubernetes consists of a family of control plane components, running on the control plane and the nodes.

GKE abstracts away the control plane.

Declare the state you want using manifest files.

